

数据分析常考面试题 101 题-业务题（45 道）、SQL（18 道）、统计学（14 道）、机器学习（24 道）

数据分析面试会重点考查候选人两方面的能力，一个是基础的数据工具能力，最常考的就是 SQL，这部分前面我们已经有系列笔记全面剖析了大厂的常考笔试题型。

考查的另一方面就是业务分析思维了，这一块主要是设定一些常见的业务场景，需要你给出对应的分析思路和方法，这一块对于没有数据分析经验的小伙伴们来说就很不友好了，很多面试的朋友碰到这类题都会很慌，无从下手，因为没做过，也不知道怎么分析啊 !!!

不过不用怕，我们收集了近几年一些大厂的面试业务题，通过这些题目的详细讲解，你应该就能掌握这类题目的解答思路了

常见的业务题

Q1、你是怎么理解数据分析的？流程如何？

数据分析是基于对业务的理解，发现业务中的问题或潜在增长点，形成分析思路，并利用数据

分析工具进行分析，给出结论和解决方案，并协调各方推动方案落地，解决问题，回到业务中去的完整闭环。

数据分析的主要流程：

观察现状

留意变化

多维、交叉分析

预测趋势

生成策略

推动落地

复盘效果

Q2、你认为数据分析的价值如何体现？

数据分析不是数据统计，不能仅仅停留在输出一份数据报告，给出一些数字，要给出结论，讲清楚数据背后的业务含义，数字的大小正负说明业务是好还是坏，有多好或者多坏。

明确数字背后的业务含义外，还要给出背后的原因，好的方面是否可以复用拓展，坏的方面原因是什么，应对方案是什么？要把方案推动落地到业务中执行，并且跟进执行的效果和复盘，螺旋上升，不断迭代优化。

Q3、指标的异常波动变化（例如日活下跌）如何分析？

确定数据来源是否可靠

确定指标的计算口径是否一致

确定是否是市场的整体趋势

描述性统计：下跌了多少

观察变化：同比下跌多少，环比下跌多少

评估变化/方差分析：跌幅是否在合理范围？

交叉分析/相关性分析：对指标进行拆解，有哪些指标可能和日活下跌有关系，有关系到什么程度

业务分析：这些指标的运营部门，是否有某些运营策略或者活动造成了这些指标的变化，间接导致日活下跌

回归分析/预测：还会跌几天？跌幅最坏到什么程度

风险/损失评估：日活下跌对产品的核心 KPI 有什么影响

制定策略：如何挽回损失/下次如何避免

Q4、注册类和活跃类指标，你会看哪个？

不同时期关注的指标不同，应区别对待。

产品初期：关注产品的快速增长，会重点关注注册类指标。

产品成熟期：关注用户的使用粘性、深度等，会重点关注留存和活跃类指标。

Q5、用 Python 设计算法实现圆周率的计算

蒙特卡洛法：一个边长为 1 的正方形和他的内切圆面积比为 $4/\pi$ ，通过随机生成 $x, y \in [0, 1]$ ，落在圆内的个数 N 除以总个数 M ，即 $N/M = \pi/4$ ，即可求出圆周率。

Q6、估算一个星巴克门店的销售额

分析 CBD 一家星巴克一个月的销售额有多少？

二八原则拆分，星巴克主要以饮品为主，饮品主要以咖啡为主。

1、从需求端估算：

饮品销量

 每天多少人喝咖啡

 CBD 一共有多少人

 喝咖啡的比例

 每人每天喝几杯

非饮品销量

2、从供给端估算：

工作日供给

忙时供给

忙时每小时多少杯

一天多少忙时

闲时供给

闲时每小时多少杯

一天多少闲时

周末供给

忙时供给

忙时每小时多少杯

一天多少忙时

闲时供给

闲时每小时多少杯

一天多少闲时

Q7、你理解的指标是什么？有哪些组成部分？

指标是有业务含义的，体现业务变化的，不是瞎拍出来的，指标是复杂的，是有特定业务场景的。

指标口径：指标的定义，业务定义比如活跃用户数：打开 APP 的用户数。

指标一般与维度一起联合分析，从不同的维度看同一个指标可能会有差异和业务上的发现。

同一个指标可能会被应用与不同的业务，需要注意保证指标口径的统一，避免造成指标的歧义和误解。

Q8、指标和维度的区别和联系

指标：用于定量评估业务好坏程度的而建立。

维度：描述指标的角度，可以理解为看问题的方面。

维度不能独立存在，一般都是搭配着指标一起分析，可以分析不同维度下的同一指标，也可以

多个维度交叉，分析某一个指标，可能会有一些业务上的发现。

Q9、什么是北极星指标？什么是虚荣指标？

北极星指标也被称为是第一关键指标，是指业务在当前阶段最为关注的一个指标，引导当前业务的发展。当然北极星指标也是一个比较汇总的指标，可以通过拆解拆分为各个子指标，更便于整体指标的实现和分析。

常见的虚荣指标包括累计用户数，累计销量、销额等，都是只见涨不见跌的虚荣指标。

Q10、什么是指标体系？如何建立？业务应用场景？

定义：

一个指标不能叫体系，多个不相关的指标也不能叫体系，一个指标体系是多个相关的指标有机结合起来，具有严格的逻辑和分层体系。

建立方法：

根据业务特点和生命周期选择第一关键指标或者叫北极星指标；

贴合业务需求将第一关键指标进行逐层拆解，可以按流程拆，按时间区域拆，按公式加减乘除拆等；

从上致下：从顶层战略或从 KPI 拆解；

从下至上：叶子指标向上汇总成整体指标；

应用场景：

监控关键指标变化趋势，判断业务走势；

逐层拆解，定位业务异常原因；

通过数据驱动业务迭代；

Q11、什么是 A/B test？核心原理和应用场景？

定义：

基于小样本的后验方法，通过设置对照组和实验组，对变量进行试验，通过假设检验对不同组

的结果进行检验，以检验变量是否对结果造成显著影响，从而选取最合理的方法。

核心逻辑：

核心是通过随机合理分流，设置对照组和实验组，通过控制变量法，在保证两组用户除待验证变量不同外，其他变量分布均一致。分别对两组施加不同的变量，观察两组用户在一段时间内的表现，通过假设检验分析结果是否有显著差异，从而判断改动是否有效可执行。

应用场景：

常应用于产品功能、算法策略迭代。

Q12、A/B test 如何合理分流？

用户分流一定要随机均匀，保证各组的用户分布一致，有分桶和分层两种方式。同一层切分为不同的桶，各桶之间是互斥的，即一个用户只能出现在一个桶里。当要同时进行多个实验时，分桶导致单个桶的流量过小，可以讲业务逻辑分为解耦的多层，不同的层共享同一批流量，这样一个用户可以进行多个实验，充分利用流量。

一种分组方式是分桶。我们直接将整体用户切割为几桶，用户只能在一个桶中。但是这种情况很不现实，因为如果我要同时上线多个实验，流量不够切怎么办？那为了达到最小样本量，我们就得延长实验周期，要是做一个实验，要几个月，相信我，你老板一定会和你聊聊人生的。

另一种方式分组方式分层。我们将业务逻辑互斥的环节分为多层，每一层都共享同一批用户，在每一层将用户随机后，使之处于不同的桶。到下一层后，对同一批用户再次随机分为几个桶。也就是说，一个用户会处于多层的多个实验中，只要多层之间互不影响，这样在保证每个实验都能用全流量切割的同时，也保证了实验数据是置信的。

Q13、如何验证 A/B test 的结果？

假设检验：一般情况下，绝对值指标用 T 检验，相对值指标用 Z 检验。

单尾/双尾检验：单尾检验的前提是我们不仅认为两组指标不同，还明确了大小，一般情况下，我们都认为实验组的效果高于基线组。而双尾检验只是认为两组指标不同，未明确大小。通常来说，我们更推荐使用双尾检验，为什么呢？因为实验本身就是一种利用数据来做决策的

方法，我们不要再人为的带入主观设想。而是用双尾检验，我们不仅能量化涨了多少，还能量化掉了多少，因为实验结果有正有负，不一定都是有效果的（正向的），还可能有负向的效果，我们也可以将有负向效果的实验记录下来，沉淀成知识库，为后期实验避坑。

Q14、什么是漏斗分析？有哪些注意的点？

通过拆分业务流程，形成关键漏斗，分析漏斗各环节的转化率/跳出率，定位问题发生的环节。漏斗分析最核心的有三点：分解流程、评估转化率和定位问题环节。

常见的漏斗有电商漏斗：首页-详情页-购物车-支付；还有用户的生命周期 AARRR；用户行为周期 AIDMA 等。

需要注意的点：

漏斗的各个环节一定是连续的，对应连续的业务流程；

漏斗的环节不宜过多，一般不超过 5 个；

一般以上一环节为基准，计算相邻两个环节的转化率；

计算每相邻环节的转化率，1-转化率即为流失率，但并不意味着转化率最低的环节就是问题环节；

漏斗可以结合其他维度看，看不同维度下的漏斗差异，进而判断是哪个因素导致了问题的出现；

Q15、数据分析这么枯燥，你为什么想做呢？

数字本身是枯燥的，但是数字背后的信息却是很有趣也很重要的。数据分析现在已经逐渐变成一种能力，一种通用的技能，通过洞察数据背后的业务价值，进而提出有指导性的建议，帮助业务发展是一件很有成就感的事。

Q16、你怎么理解统计学？生活中统计学应用举例？

作为数据分析师，所有的事实都应该由数据说话，但数据本身是没有意义的，更不会告诉我们数据的高低好坏，我们需要基于数据给出科学可靠的结论，这个过程就不得不用到统计学知识。好的到什么程度算好，坏到什么程度算坏，高到什么程度算是有显著性差异，这些都不是我们个人凭主观感觉就能够给出来的，需要经过统计学的知识谨慎给出结论，这样才不至于给出

错误的结论而导致错误的决策。

如果用平均值算财富的话，我和马云的财富平均下来，我也是亿万富翁，这显然不科学，因为个人财富并不服从正态分布，我们用平均值无法代表一般水平，这个时候如果使用中位值，看全国排名 50% 的人的收入，可能更足以说明大家的一般水平是如何。

原文链接：<https://blog.csdn.net/licent2011/article/details/124397128>

Q17、什么是 EDA(Exploratory Data Analysis)?

EDA (Exploratory Data Analysis) 即数据探索性分析，需要对数据集中的变量进行统计和分布描述、了解变量间的相互关系，从整体上了解数据集的数据特征。探索性分析要对调查总体所有变量的有关数据进行统计性描述，主要包括数据的频数分析、集中趋势分析、离散程度分析、分布以及一些基本的统计图形。

①数据的频数分析。在数据的预处理部分，利用频数分析和交叉频数分析可以检验异常值

②数据的集中趋势分析。用来反映数据的一般水平，常用的指标有平均值、中位数和众数等。

③数据的离散程度分析。主要是用来反映数据之间的差异程度，常用的指标有方差和标准差。

④数据的分布。在统计分析中，通常要假设样本所属总体的分布属于正态分布，因此需要用偏度和峰度两个指标来检查样本数据是否符合正态分布。

⑤数据的相关性分析。探索不同变量之间的相关性，可以使用相关性图进行展示，表示各变量之间的相关程度，为后续机器学习的特征选择提供依据。

Q18 计算西瓜视频内容好评率

1) 需求

西瓜视频近期开展了“ 2020 百大人气创作者” 优质内容扶持项目，鼓励用户产出优质的视频

内容。现需要统计 2020 年 11 月 01 日至 2020 年 11 月 30 日期间创作的视频中，“科技”大类下“数码测评”子类的视频好评率（好评率=好评数/视频观看次数），写出 sql 语句进行查询。

用户观看视频后的评价详情表：content_action_info

id（视频 id，主键）

create_time（创建时间，格式‘2020-11-01’）

user_id（观看者 id）

content_id（视频 id，外键）

content_action（视频评价，包括‘点赞’，‘差评’，‘无评价’）

视频详情表：dim_content

content_id（外键）

creator_id（创作者 id）

content_category（商品类目）

content_sub_category（品牌名称）

2）解题思路

题目很简单，我们只需要明确好评率的计算公式，然后先定特定的大类和子类即可。

1.好评率=好评数/视频观看次数。

2.要求计算“科技”大类下“数码测评”子类的视频好评率，则使用 where 条件筛选指定大类和子类的数据。

3.因为限定条件要指定视频内容的类别，所以要两个表做关联。

代码如下：

```
select

count(1) as all_action

sum(case when content_action='点赞' then 1 else 0 end) as

like_action

sum(case when content_action='点赞' then 1 else 0 end)/count(1) as

like_rate

from content_action_info as a

join dim_content as b

on a.content_id=b.content_id

where b.content_category='科技'

and b.content_sub_category='数码测评'

and a.create_time between '2020-11-01' and '2020-11-30';
```

Q19 使用 AB Test 评估算法效果

1) 需求

某个购物 APP 最近优化了“猜你喜欢”模块的推荐算法，希望进一步提升推荐的精准度，进而提升销售额。现在需要通过 AB Test（50%用户保留原先推荐算法的为控制组，50%用户使用新的推荐算法为实验组）来对新的推荐效果进行评估。假设你是此次实验的数据分析师，请问你会如何评估控制组和实验组的表现？请按重要性列出最重要的三个指标并给出你的分析过程。

2) 解题思路

指标：推荐商品的销售额、推荐商品点击率、推荐商品的转化率

分析过程：

1. 设原假设为使用新的推荐算法后上述指标降低或不变，备择假设为使用新的推荐算法后上

述指标增加。

2. 选择显著性水平临界值为 5%，并根据指标的预期提升确定样本量和试验周期。
3. 样本合理分流，AB 测试上线，采集数据。
4. 使用 T 检验，计算 P 值，进行效果验证。
5. 分析结论，如果 P 值小于 5%，那么原假设不成立，备择假设成立，即使用新算法后指标提升。

反之无法推翻原假设，不能证明使用新算法后指标提升。

Q20 如何评估一场活动的效果

1) 需求

某 APP 近期上线了一个拉新活动，并在各个渠道进行了推广投放，活动结束后，作为数据分析师，你如何评估这场活动的效果？

2) 解题思路

如果要对一个活动效果进行分析，无非是要回答以下 3 个问题：

- 1、活动效果怎么样？要不要继续做？
- 2、如果可以继续做，活动的做的好的方面是哪些？问题或者瓶颈环节在哪？
- 3、针对问题环节的改进方案是什么？

具体展开来讲，可以从以下几个方面

1、活动关键指标达成分析

活动关键核心指标达成情况，比如拉新多少用户，达成多少 GMV?ROI 如何？

2、活动关键流程漏斗分析

活动的关键流程是什么？以及各个流程的漏斗分析，定位问题发生的环节。

3、活动的渠道、用户分析

活动在哪些渠道推送？活动推送给哪些用户？用户画像是啥样的？各渠道用户的质量/ROI 如何？

4、活动策略、节奏分析

活动玩法的裂变效果如何？利益点是否有吸引力？活动整个过程节奏把控如何，前期预热、中期爆发和尾期是否过短/过长，运营应该在何时进行适当干预。

Q21 用户网购行为分析

1) 需求：

分析有订单事务表：orders

加购事务表：addcart

请用 SQL 代码分析用户对商品的行为特征，包括：已购买，购买未加购，加购未购买，加购且购买。

2) 解题思路：

1.需要得到 user_id 和 item_id 的购买和加购的组合情况，使用这两个主键进行关联。

2.根据是否可以关联来判断是否购买和是否加购。

3.使用 full join 外连接，MySQL 不支持 full join，可以使用左连接和右连接后使用 union 即可。

代码如下：

```

SELECT  a.user_id,

        a.item_id,

        1 AS `已购买`,

        CASE WHEN b.item_id IS NULL THEN 1 ELSE 0 END AS `购买未
加购`,

        0 AS `加购未购买`,

        CASE WHEN a.item_id = b.item_id THEN 1 ELSE 0 END AS `加
购且购买`,

FROM    orders a

LEFT JOIN

        addcart b

ON      a.user_id = b.user_id

AND     a.item_id = b.item_id

UNION

SELECT  a.user_id,

        a.item_id,

        CASE WHEN b.item_id IS NOT NULL THEN 1 ELSE 0 END AS `已
购买`,

        0 AS `购买未加购`,

        CASE WHEN b.item_id IS NULL THEN 1 ELSE 0 END AS `加购未
购买`,

        CASE WHEN a.item_id = b.item_id THEN 1 ELSE 0 END AS `加
购且购买`,

FROM    addcart a

LEFT JOIN

```

Q22 销售额下降了如何分析？

1) 需求

这是一道非常经典的业务题。

如果我们发现，某店铺 2020 年 11 月的销售额同比去年 11 月下降了 60%，如果你是这块业务的数据分析师，你会如何分析？请写出你的分析思路框架。

2) 解题思路：

1、先确认数据来源是否可靠，指标口径是否一致，很多时候，大家只是道听途说，讲的不是同一个事情，埋头就开始分析，反而浪费了时间精力，需要首先确认问题是否真实存在。

2、如果数据来源可靠，指标口径一致，先看外部市场环境，是否是整体市场大盘在下降，受到大盘的影响。

3、如果不是外部因素，看内部问题，对内部因素进行指标成分拆解，可以按照以下的方式进行拆解。

4、具体可能的拆解方式如下：

销售额 = 访客数 * 下单率 * 客单价 - 退货金额

访客数 = 新客户 + 老客户，新客户 = 渠道的流量 * 转化率

下单率 = 页面访问量 * 转化率

订单取消率 = 订单取消数/订单总数

5、针对指标的变化进行提建议，定位到具体的原因后，针对性的给出改进策略方案，如开展促销活动、商品。

Q23 估算一个星巴克门店的销售额

1) 需求

某 CBD 楼下有一个星巴克门店，面积约为 40 平方米。请预估该门店每周的营业额。这其实是一个非常经典的费米估算问题，出现这类问题，并不是要求你要十分精确的计算出最终数据。而是考查你分析和拆解问题的思维能力。

2) 解题思路：

费米思想的两个步骤：

1.把问题进行拆解，直到拆解到不能拆解需要估算的基本问题

2.对基本问题的估算，不要估算其本身，而是先估算其上下界，然后在上下界的范围内估算出数值。

营业额 = 消费人数 * 平均消费金额（区分工作日和周末）

消费人数 = 营业时间 * 单位时间消费人数（区分高峰时段 + 低峰时段）

3) 估算：

每天按照 12h 营业时间计算，其中客流量大有 4h，客流量小 8h（正常估）。

休息日平均营业额/4 工作日平均营业额（正常估）。

工作日流量大的时候，每 3 分钟接高一人次，人均消费 30 元（高估）。

一个店 40 平米，我们假设 50%的比例，那就是 20 低平米，可以站下 10 个人（低估）。

那这样工作日高峰时间段每小时消费人数 = $1\text{h}/3\text{min} * 10 = 20 * 10 = 200$ 个，非高峰估算 60 个。

那工作日一天 营业额 = $4\text{h} * 200\text{个} * 30\text{元} + 8\text{h} * 60\text{个} * 30\text{元} = 38400$

一周的营业额 = $38400 * 5 + 38400/4 * 2 = 211200$ 。

Q24 什么是同比、环比，意义是什么？

- 环比：与相连续的上一个统计周期进行对比，环比增长速度 = $(\text{本期数} - \text{上期数}) \div \text{上期数} \times 100\%$ ，反映本期比上期增长了多少，如 2021 年 12 月和 2021 年 11 月对比，环比的好处是可以更直观的表明阶段性的变换，但是会受季节性因素影响。
- 同比：是对去年同期的一个统计阶段进行对比，同比增长率 = $(\text{本期数} - \text{同期数}) \div \text{同期数} \times 100\%$ ，如今年的这个月和去年的这个月，同比的好处是可以排除一部分季节因素。

Q25 什么是相关性分析？相关和因果的区别是什么？

相关性的前提是各个变量之间是相互独立的，业务上来说，每个指标之间是没有影响的，相关性系数：0.5 以下相关性较弱，0.5 - 0.8 中度相关，大于 0.8 较强相关。

如果 A 和 B 相关，会有以下几个推论：

A 导致 B：充分条件

B 导致 A：必要条件

C 导致 A 和 B：A 和 B 同源

A 和 B 如果有因果：充分且必要条件

相关和因果的区别：

相关：两个变量或多个变量之间的相互影响程度；核心点：变量与变量之间互为相关，没有先后顺序；

因果：前一个事件对后一个事件的作用的关系及强度，核心点：有严格的先后顺序（变化可能会有延迟），如蝴蝶效应。

好玩的题目：

$Y=0.5X$ y 和 x 是相关还是因果？

$y=0.5X+0.5z$ y 和 x 是相关还是因果

答案：都是相关

因果的前提是：有一个东西是先发生，而相关是没有先后顺序的，所以上述 2 个都是相关，而不是因果。

Q26 什么是聚类？业务应用场景？常见算法？

定义：

将相似的对象，将对象的特征进行抽象，通过算法将特征相似的对象化为一类，是一种无监督机器学习算法。

- 考点 1：对于聚类，如何确定分类的数量 N ；
- 考点 2：如何选择进行聚类的特征，比如要区分男女，有喉结的就是男生，留长发的就是女生；

应用场景：

- 个性化推荐（电商）：相似用户行为相似，会分到某个类，如用户分层；
- 用户画像：基于用户的购买偏好、消费能力进行用户画像；

常见算法：

Kmeans 聚类

DBSCAN 聚类

Q27 什么是分类？业务应用场景？常见算法？

定义：

学习已有分类样本的特征，对新数据进行划分，是一种有监督的机器学习算法，分类是明确的，是有历史样本可学习的。

应用场景：

- 互联网金融用户信用等级分类
- 垃圾邮件分类

常见算法：

- 逻辑回归
- SVM
- 贝叶斯
- 决策树
- KNN
- XGboost

Q28 什么是回归？业务应用场景？常见回归算法？

定义：

两个或多个变量之间是否相关，相关强度，并建立数学模型，定量评估

- 人话：我和你有没有关系，什么关系，深到什么程度。
- 本质：找到一条之间最合适的平均线，让线的附近的点分布均匀

应用场景：

- 运营推广中，是不是花的钱越多，买的流量越大，品类越丰富，用户活跃越高，那么，多到什么程度、大到什么程度、丰富到什么程度、用户的活跃最高，留存最高；
- 智慧城市的交通，预测交通拥塞程度：自变量：时间段、商业指数、住宅指数、城区指数、道路指数，因变量：两个连续信号灯之间道路的通行时间。

常见回归算法：

- 线性回归
- lasso 回归
- ridge 回归
- 树回归

Q29 时间序列预测的原理是什么？有哪些应用场景？

原理：

当自变量是时间时，且数据在时间上呈现出一定的规律，那么这种情况一般都可以使用时间序列预测接下来一段时间的数据走势。这个规律表现为：数据整体变化的趋势（可以理解为一条表征趋势的直线）、季节性（可以理解为按照一定的周期重复出现的模式）和随机性（可以理解为在零附近毫无规律的白噪声）组成，时间序列就是将数据按照这三个部分分别拆解，再基于历史数据进行组合预测。

应用场景：

一般应用于年度的 KPI 预测，产品的活跃用户数趋势，羽绒服的销量等。

Q30 时间序列预测需要注意的点？和回归有何区别？

需要注意的点：

时间间隔是固定的；

最近的数据对于预测影响程度越大；

预测是有季节性的，这里的季节性不一定是春夏秋冬，凡是以一定周期重复出现的都可以称为有季节性；

Q31 回归预测和时间序列预测的区别：

- 回归是自变量对于因变量的趋势，用以表征自变量和因变量之间的定量关系，一般来说自变量和因变量只能是连续的数据；
- 时间序列预测的自变量可以是任何数据，包括时间，只要数据呈现出周期性的趋势；
- 回归不能做季节性的预测；

Q32. 不用任何公开参考资料，估算今年新生儿出生数量。

- 采用两层模型（人群画像 x 人群转化）：新生儿出生数 = $\sum$ 各年龄层育龄女性数量 * 各年龄层生育比率

（一般面试中采用这种方法，即费米估计问题，可以参考《这也能想到？——巧妙解答无厘头问题》）

- 从数字到数字：如果有前几年新生儿出生数量数据，建立时间序列模型（需要考虑到二胎放开的突变事件）进行预测
- 找先兆指标，如婴儿类用品的新增活跃用户数量 X 表示新生儿家庭用户。Xn/新生儿 n 为该年新生儿家庭用户的转化率，如 X2007/新生儿 2007 为 2007 年新生儿家庭用户的转化率。该转化率会随平台发展而发展，可以根据往年数量推出今年的大致转化率，并根据今年新增新生儿家庭用户数量推出今年估计的新生儿数量。

Q33. 如果次日用户留存率下降了 5%该怎么分析？

- 首先采用“两层模型”分析：对用户进行细分，包括新老、渠道、活动、画像等多个维度，然后分别计算每个维度下不同用户的次日留存率。通过这种方法定位到导致留存率下降的用户群体是谁。

- 对于目标群体次日留存下降问题，具体情况具体分析。具体分析可以采用“内部-外部”因素考虑。

a. 内部因素分为获客（渠道质量低、活动获取非目标用户）、满足需求（新功能改动引发某类用户不满）、提活手段（签到等提活手段没达成目标、产品自然使用周期低导致上次获得的大量用户短期内不需要再使用等）；

b. 外部因素采用 PEST 分析（宏观经济环境分析），政治（政策影响）、经济（短期内主要是竞争环境，如对竞争对手的活动）、社会（舆论压力、用户生活方式变化、消费心理变化、价值观变化等偏好变化）、技术（创新解决方案的出现、分销渠道变化等）。3. 卖玉米如何提高收益？价格提高多少才能获取最大收益？

- 收益 = 单价 * 销售量，那么我们的策略是提高单位溢价或者提高销售规模。

○ 提高单位溢价的方法：

（1）品牌打造获得长期溢价，但缺陷是需要大量前期营销投入；

（2）加工商品占据价值链更多环节，如熟玉米、玉米汁、玉米蛋白粉；重定位商品，如礼品化等；

- (3) 价格歧视，根据价格敏感度对不同用户采用不同定价。销售量=流量 × 转化率，上述提高单位溢价的方法可能对流量产生影响，也可能对转化率产生影响。
- 收益 = 单价 × 流量 × 转化率，短期内能规模化采用的应该是进行价格歧视，如不同时间、不同商圈的玉米价格不同，采取高定价，然后对价格敏感的用户提供优惠券等。

Q34. 类比到头条的收益，头条放多少广告可以获得最大收益，不需要真的计算，只要有个思路就行。

- 收益 = 出价 × 流量 × 点击率 × 有效转化率，放广告的数量会提高流量，但会降低匹配程度，因此降低点击率。最大收益是找到这个乘积的最大值，是一个有约束条件的最优化问题。同时参考价格歧视方案，可以对不同的用户投放不同数量的广告。

Q35.APP 激活量的来源渠道很多，怎样对来源渠道变化大的进行预警？

- 如果渠道使用时间较长，认为渠道的 app 激活量满足一个分布，比较可能是正态分布。求平均值和标准差，对于今日数值与均值差大于 3/2/1 个标准差的渠道进行预警。
- 对于短期的新渠道，直接与均值进行对比。

Q36.用户刚进来 APP 的时候会选择属性，怎样在保证有完整用户信息的同时让用户流失减少？

- 采用技术接受模型（TAM）来分析，影响用户接受选择属性这件事的主要因素有：

技术接受模型提出了两个主要的决定因素：

- ①感知的有用性(perceived usefulness)，反映一个人认为使用一个具体的系统对他工作业绩提高的程度；
- ②感知的易用性(perceived ease of use)，反映一个人认为容易使用一个具体的系统的程度。

(1) 感知有用性：

a. 文案告知用户选择属性能给用户带来的好处

(2) 感知易用性:

a. 关联用户第三方账号 (如微博), 可以冷启动阶段匹配用户更有可能选择的属性, 推荐用户选择。

b. 交互性做好。

(3) 使用者态度: 用户对填写信息的态度

a. 这里需要允许用户跳过, 后续再提醒用户填写

b. 告知用户填写的信息会受到很好的保护

(4) 行为意图: 用户使用 APP 的目的性, 难以控制

(5) 外部变量: 如操作时间、操作环境等, 这里难以控制

Q37.如何识别作弊用户 (爬虫程序, 或者渠道伪造的假用户)

● 分类问题可以用机器学习的方法去解决, 下面是我目前想到的特征:

(1) 渠道特征: 渠道、渠道次日留存率、渠道流量以及各种比率特征

(2) 环境特征: 设备 (一般伪造假用户的工作坊以低端机为主)、系统 (刷量工作坊一般系统更新较慢)、wifi 使用情况、使用时间、来源地区、ip 是否进过黑名单

(3) 用户行为特征: 访问时长、访问页面、使用间隔、次日留存、活跃时间、页面跳转行为 (假用户的行为要么过于一致, 要么过于随机)、页面使用行为 (正常用户对图片的点击也是有分布的, 假用户的行为容易过于随机)

(4) 异常特征: 设备号异常 (频繁重置 idfa)、ip 异常 (异地访问)、行为异常 (突然大量点击广告、点赞)、数据包不完整等 8.怎么做恶意刷单检测?

● 分类问题用机器学习方法建模解决, 我想到的特征有:

(1) 商家特征: 商家历史销量、信用、产品类别、发货快递公司

(2) 用户行为特征: 用户信用、下单量、转化率、下单路径、浏览店铺行为、支付账号

(3) 环境特征 (主要是避免机器刷单): 地区、ip、手机型号等

(4) 异常检测: ip 地址经常变动、经常清空 cookie 信息、账号近期交易成功率上升等

(5) 评论文本检测: 刷单的评论文本可能套路较为一致, 计算与已标注评论文本的相似度作为特征

(6) 图片相似度检测: 同理, 刷单可能重复利用图片进行评论

Q38.一个网站销售额变低，你从哪几个方面去考量？

- 首先要定位到现象真正发生的位置，到底是谁的销售额变低了？这里划分的维度有：
 - a. 用户（画像、来源地区、新老、渠道等）
 - b. 产品或栏目
 - c. 访问时段定位到发生未知后，进行问题拆解，关注目标群体中哪个指标下降导致网站销售额下降：
 - a. 销售额=入站流量 × 下单率 × 客单价
 - b. 入站流量 = $\sum$ 各来源流量 × 转化率
 - c. 下单率 = 页面访问量 × 转化率
 - d. 客单价 = 商品数量 × 商品价格确定问题源头后，对问题原因进行分析，如采用内外部框架：
 - a. 内部：网站改版、产品更新、广告投放
 - b. 外部：用户偏好变化、媒体新闻、经济环境、竞品行为等.

Q39.用户流失的分析，新用户流失和老用户流失有什么不同？

(1) 用户流失分析：

- 两层模型：细分用户、产品、渠道，看到底是哪里用户流失了。注意由于是用户流失问题，所以这里细分用户时可以细分用户处在生命周期的哪个阶段。
- 指标拆解：用户流失数量 = 该群体用户数量 × 流失率。拆解，看是因为到了这个阶段的用户数量多了（比如说大部分用户到了衰退期），还是这个用户群体的流失率比较高
- 内外部分析：
 - a. 内部：新手上手难度大、收费不合理、产品服务出现重大问题、活动质量低、缺少留存手段、用户参与度低等
 - b. 外部：市场、竞争对手、社会环境、节假日等

(2) 新用户流失和老用户流失有什么不同：

- 新用户流失：原因可能有非目标用户（刚性流失）、产品不满足需求（自然流失）、产品难以上手（受挫流失）和竞争产品影响（市场流失）。
- 新用户要考虑如何在较少的数据支撑下做流失用户识别，提前防止用户流失，并如何对有效的新用户进行挽回。老用户流失：原因可能有到达用户生命周期衰退期（自然流失）、过度拉升 arpu 导致低端用户驱逐（刚性流失）、社交蒸发难以满足前期用户需求（受挫流失）和竞争产品影响（市场流失）。

老用户有较多的数据，更容易进行流失用户识别，做好防止用户流失更重要。当用户流失后，

要考虑用户生命周期剩余价值，是否需要挽回。

参考@王玮 的回答：如何进行用户流失原因调研？ 2107 关注 · 21 回答问题

Q40.GMV 升了 20%怎么分析？

（我嘞里啪啦分析了一通面试官笑嘻嘻地告诉我是数据错了，因为面试较紧张没有意识到这个问题，现在想想真是个大坑啊）

- 参考该面试者经验，应该先估算一下数字有没有问题

- 同样的套路：

- （1）两层模型：进行用户群体、产品、渠道细分，发现到底是谁的 GMV 提升了

- （2）指标拆解：将 GMV 拆解成乘法模型，如 $GMV = \text{广告投放数量} \times \text{广告点击率} \times \text{产品浏览量} \times \text{加入购物车率} \times \text{交易成功率} \times \text{客单价}$ ，检查哪一步有显著变化导致了 GMV 上升

- （3）内外部分析：

- a. 内部：网站、产品、广告投放、活动等

- b. 外部：套 PEST 等框架也行，或者直接分析也行，注意 MEMC 即可这一题要注意，GMV 流水包括取消的订单金额和退货/拒收的订单金额，还有一种原因是商家刷单然后退货，虽然 GMV 上去了，但是实际成交量并没有那么多。

Q41.如果现在有个情景，我们有一款游戏收入下降了，你怎么分析。

- 两层模型：细分用户、渠道、产品，看到底是哪里的收入下降了

- 指标拆解：收入 = 玩家数量 * 活跃占比 * 付费转化率 * 付费次数 * 客单价

- 进一步细分，如玩家数量 = 老玩家数量 * 活跃度 + 新玩家数量 * 留存率等。然后对各个指标与以往的数据进行对比，发现哪些环节导致收入下降原因分析：

- a. 内部：产品变化、促活活动、拉新活动、定价策略、运营策略、服务器故障等

- b. 外部：用户偏好变化、市场环境变化、舆论环境变化、竞争对手行为、外部渠道变化等
如何提高：基于乘法模型，可以采用上限分析，从前往后依次将指标提升到投入足够精力（假设优先分配人力、经费与渠道）后的上限，然后分析“收入”指标的数值提升。找到数值提升最快的那个阶段，就是我们提高收入的关键任务

Q42.现在有一个游戏测试的环节，游戏测试结束后需要根据数据提交一份 PPT，这个 PPT 你会如何安排？包括什么内容？

这里可以套 AARRR 模型：

获取用户（Acquisition）

提高活跃度（Activation）

提高留存率（Retention）

获取收入（Revenue）

自传播（Refer）

- 获取：我们的用户是谁？用户规模多大？
- a. 用户分层激活：游戏是否吸引玩家？哪个渠道获取的用户有质量（如次日留存高、首日停留时间长等）？
- 留存：用户能否持续留存？哪些用户可以留存？
- 转化：用户的游戏行为如何？能否进行转化？能否持续转化？
- 自传播：用户是否会向他人推荐该游戏？哪种方式能有效鼓励用户推荐该游戏？传播 k 因子是否大于 1？

Q43.比如你对楼市不熟悉，现在要你去做一个像 58 同城之类的，卖房的中介，电商，你会如何进行分析。

（1）商业模式分析：中介做的是双边市场生意，通过解决市场信息不对称下的信息流动问题，降低买方和卖方的交易成本，从而创造盈利空间

（2）需求分析：

- a. 买方需求分析：低价买好房，对时间的需求有快的和慢的
- b. 卖方需求分析：房子卖高价，对时间的需求有快的和慢的

（3）进入条件分析（套 SWOT 分析法（态势分析法）也行）：

- a. 自身条件
- b. 竞争对手
- c. 市场增长规模

（4）进入策略分析：

- a. 自身目标
- b. 目标拆解

- c. 分析目标达成的可能性，预估将来一段时间，好/正常/坏条件下的目标达成情况
- d. 得出结论：是否进入该市场

Q44. 某业务部门在上周结束了为期一周的大促，作为业务对口分析师，需要你对活动进行一次评估，你会从哪几方面进行分析？

- (1) 确定大促的目的：拉新？促活？清库存？
- (2) 根据目的确定核心指标。
- (3) 效果评估：
 - a. 自身比较：活动前与活动中比较
 - b. 与预定目标比
 - c. 与同期其它活动比
 - d. 与往期同类活动比
- (4) 持续监控：
 - a. 检查活动后情况，避免透支消费情况发生
 - b. 如果是拉新等活动，根据后续数据检验这批新客的质量

Q45 两道蛮好的鹅厂题

<https://www.zkxjob.com/19754>

基础的数据工具

SQL

本小节仅搜集常问的几道 SQL 题，更多的可以移步到 LeetCode 上面 SQL 里面去练习。重要的事情说三遍。点击链接 <https://leetcode.cn/study-plan/sql/>

1. 求连续时间问题

这类题的核心是：分组排序，用时间减去排序，如果连续的话他们的差会是相同值。

记住下面的表，按照提供的三步思路理清楚里面的数据逻辑，连续时间问题再不会难到你

日期	排序	diff
1月1号(1)	1	0
1月2号(2)	2	0
1月3号(3)	3	0
1月7号(7)	4	3
1月8号(8)	5	3

思路：

求解连续天数是面试时非常常考的问题，这类问题有非常巧妙的统一解法，分为 3 步：

1、为日期排序

· - row_number() over (partition by use_id order by date) as rank

2、求日期和排序的差值 (diff)

· - 因为日期存在周期，会存在跨月、跨年的情况，故使用 datediff 最保险

· date_diff(date, rank) as diff

3、求 diff 出现最多的次数

· - max(count(diff))

<问题 1>

· 日活表求每个用户一周活跃天数

· 日活表求每个用户一周内最大连续活跃天数

date	user_id	time
20210101	1	2021-01-01 09-01:00 00 00
20210101	1	2021-01-01 11-02:00 00 00
20210101	2	2021-01-02 10-08:00 00 00
20210102	2	2021-01-02 15-05:00 00 00

-- 日活表求每个用户一周活跃天数

```
SELECT  user_id,

        count(1)

FROM    (

        SELECT  user_id,

                datefrom tablewhere TIMESTAMPDIFF(DAY, TIME,

NOW()) <= 7

        GROUP BY

                user_id,

                date

        )

GROUP BY

        user_id
```

```

-- 日活表求每个用户一周内最大连续活跃天数-- 求出现次数最多的 diff

SELECT  user_id,

        MAX(count)

FROM      (

,

        -- 算出登陆日期与排序的 diff      select user_id

        DATE_SUB(date, RANK) AS gap,

        count(1) AS count

        FROM      (

,

                                -- 为每个用户的登录，按时间排

序      select user_id

                                date,

                                row_number() OVER (

                                PARTITION BY

                                user_id

                                ORDER BY

                                date

                                ) AS RANK

                                FROM      (

                                -- 选出每个用户最近 7 天的记录，同

时每个用户每天只保留一条记

录      select distinct user_id, date

                                FROM      TABLE

                                WHERE      TIMESTAMPDIFF(DAY,

NOW(), date) <= 7

```

这里用到了 row_number() over (partition by [] order by [])函数、TIMESTAMPDIFF () 函数和 date_sub()函数，解释如下：

- row_number() over (partition by [] order by [])是一种非常常用的分组排序函数，可以将排序的结果输出成一个新的列，partition by []是指按什么字段分组，order by [] 用于设置按什么顺序，row_number() 用于设置排序的方式，除了 row_number()外还有 rank 和 dense_rank

- TIMESTAMPDIFF () 语法规则是(unit , begin , end) , 返回 begin-end 的结果，其中 begin 和 end 是 DATE 或 DATETIME 表达式，unit 参数是确定(end-begin)的结果的单位，表示为整数，取值可以是秒 second，分钟 minute，小时 hour 和天 day 等

- date_sub() 语法规则 (time , int) , 将时间减去 n 天

<问题 2>

给定一张用户签到表 user_attendence，表中包含三个字段，分别是用户 ID：【user_id】，日期：【date】，是否签到：【is_sign_in】，0 否 1 是

user_id	date	is_sign_in
1	2021-01-02	0
1	2021-01-03	1

- 计算截至当前，每个用户已经连续签到的天数

```
SELECT  user_id,

        TIMESTAMPDIFF(DAY, el, NOW()) AS contDays

FROM    (

        SELECT  user_id,

                MAX(date) AS el

        FROM

        WHERE    is_sign_in = 0

        GROUP BY

                user_id

    )
```

思路：选出最新没签到的日子，用当前时间减去该日子得到已经连续签到的次数

2. 时间间隔问题举例

- 求每个用户相邻两次浏览时间之差小于 3min 的次数
- 求用户相邻两次提交时间小于 1h 的次数
- 求用户购买同一商品的间隔时长

这类题的核心是：使用分组排序函数，对同一用户的购买行为，根据时间先后顺序排序；使用错位相减，求出两次相邻时间。

记住下面的表，按照提供的三步思路理清楚里面的数据逻辑，连续时间问题再不会难到你

用户编号	操作时间	日期排序	时间差
a	1月1日	1	1
a	1月2日	2	3
a	1月5日	3	-
b	1月1日	1	1
b	1月2日	1	-

思路：

求解时间间隔是面试时非常常考的问题，这类问题有非常巧妙的统一解法，分为 3 步：

1、为日期排序

· -row number() partition by 用户编号 order by 操作时间 as 日期排序

2、错位相减，求日期和排序的差值 (diff)

· 用错位相减求来实现 “相邻” 要求

```
select b.日期排序-a.日期排序 as 时间差  
  
from a  
  
left join b on (a.日期排序 = b.日期排序-1)
```

· -因为日期存在周期，会存在跨月、跨年的情况，故使用 datediff 最保险

· date_diff(date, rank) as diff

3、根据题目要求，求出相应指标

· max(时间差)

· count(*)

· min(时间差)

<问题>

· 求每个用户相邻两次浏览时间之差小于 3min 的次数

· 字段：分别是用户 id :【user_id】和用户访问时间【user_time】

user_id	user_time
a	20210101
a	20210102
a	20210103
b	20210101

```

select user_id
      ,count(*) as numfrom(
      select a.user_id as user_id
            ,b.user_time-a.user_time as time_diff -- 求出相邻时间
差      from
      (
      select user_id
            ,user_time
            -- 按时间排
序      ,row_number() over (partition by user_id order
by user_time) as rank
            from table) a
      left join
      (
      select user_id
            ,user_time
            -- 按时间排
序      ,row_number() over (partition by user_id order
by user_time) as rank
            from table) b on (a.user_id=b.user_id and a.rank=b.rank-1)) -- 错位相减关键步骤 where time_diff<=3group by user_id;

```

<延伸问题>

· 求同一个用户购买同一个商品相邻两次的时间间隔

思路：只需要更细粒度的分组即可，使用 partition by 对多列进行分组排序

row_number() over(partition by [column1],[column2] order by [column3]) as xxx

3. Hive Sql 求累加问题

数据分析笔试中累加问题是非常常见的考题，今天我们用一个函数来搞定它

```
sum over(partition by 分组列 order by 排序列 rows between 开始位置  
preceding and 结束位置 following)
```

其中'开始位置'和'结束位置'可配置参数：数据、current、UNBOUNDED

问题 1

数据集有三列：userid，month，count，统计每个用户截止到当月为止的最大单月访问次数和累计到该月的总访问次数

```
select

    Userid

    ,month

    ,max(count) over (partition by userid,month) as max_cnt

    ,sum(count) over (partition by userid,month) as sum_cnt

from

    Table
```

问题 2

有这样一张表，记录了每个月的营业额，数据如下：

表名：test

字段：month | money

统计截止到每个月份的营业总额

-- 方法 1

```
select sum(money) from test where month='201901'
```

```
union all
```

```
select sum(money) from test where month<='201902'
```

```
union all
```

```
select sum(money) from test where month<='201903'
```

```
...
```

```
"""
```

优点：这是最容易想到的解决方法，实现起来又方便

缺点：此方法虽能解决问题，可略显笨重，且同一张表扫描次数过多，执行速度缓慢，不可取。

```
"""
```

-- 方法 2

```
select
```

```
    sum(case when month=201901 then money else null end) as
```

```
sum_money_01,
```

```
    sum(case when month<=201902 then money else null end) as
```

```
sum_money_02,
```

```
    sum(case when month<=201903 then money else null end) as
```

```
sum_money_03,
```

```
    sum(case when month<=201904 then money else null end) as
```

```
sum_money_04,
```

```
    sum(case when month<=201905 then money else null end) as
```

```
sum_money_05,
```

```
    sum(case when month<=201906 then money else null end) as
```

4. Sql 窗口分析函数【lead、lag 详解】

lag : 用于统计窗口内往上第 n 行值

lead : 用于统计窗口内往下第 n 行值

lag 和 lead 有三个参数，第一个参数是列名，第二个参数是偏移的 offset，第三个参数是超出记录窗口时的默认值。

```
lag(列名,1,0) over (partition by 分组列 order by 排序列 rows between  
开始位置 preceding and 结束位置 following)
```

案例 1

有一个日志登陆列表，获取用户在某个页面停留时长

```
+-----+-----+-----+--+
| userid          |          time          | url          |
+-----+-----+-----+--+
| Peter           | 2015-10-12 01:10:00    | url1        |
| Peter           | 2015-10-12 01:15:10    | url2        |
| Peter           | 2015-10-12 01:16:40    | url3        |
| Peter           | 2015-10-12 02:13:00    | url4        |
| Peter           | 2015-10-12 03:14:30    | url5        |
| Marry           | 2015-11-12 01:10:00    | url1        |
| Marry           | 2015-11-12 01:15:10    | url2        |
| Marry           | 2015-11-12 01:16:40    | url3        |
| Marry           | 2015-11-12 02:13:00    | url4        |
| Marry           | 2015-11-12 03:14:30    | url5        |
+-----+-----+-----+--+
```

代码

```
select

    userid

    ,time

    ,UNIX_TIMESTAMP(lead(time,1) over(partition by userid order
by time),'yyyy-MM-dd HH:mm:ss') - UNIX_TIMESTAMP(time,'yyyy-MM-dd
HH:mm:ss') as period

    ,url

from

    user_log
```

结果

```

+-----+-----+-----+-----+
-+-----+---+

|  userid  |          stime          |          etime          |  period
|   url   |
+-----+-----+-----+-----+
-+-----+---+

|  Marry   | 2015-11-12 01:10:00    | 2015-11-12 01:15:10    | 310
| url1    |
|  Marry   | 2015-11-12 01:15:10    | 2015-11-12 01:16:40    | 90
| url2    |
|  Marry   | 2015-11-12 01:16:40    | 2015-11-12 02:13:00    | 3380
| url3    |
|  Marry   | 2015-11-12 02:13:00    | 2015-11-12 03:14:30    | 3690
| url4    |
|  Marry   | 2015-11-12 03:14:30    | NULL                    | NULL
| url5    |
|  Peter   | 2015-10-12 01:10:00    | 2015-10-12 01:15:10    | 310
| url1    |
|  Peter   | 2015-10-12 01:15:10    | 2015-10-12 01:16:40    | 90
| url2    |
|  Peter   | 2015-10-12 01:16:40    | 2015-10-12 02:13:00    | 3380
| url3    |
|  Peter   | 2015-10-12 02:13:00    | 2015-10-12 03:14:30    | 3690
| url4    |
|  Peter   | 2015-10-12 03:14:30    | NULL                    | NULL
| url5    |

```

案例 2

寻找至少连续出现 3 次的数字

```
+-----+-----+
| id           | num |
+-----+-----+
| 1           | 1   |
| 2           | 1   |
| 3           | 1   |
| 4           | 1   |
| 5           | 2   |
| 6           | 2   |
| 7           | 3   |
| 8           | 3   |
| 9           | 3   |
| 10          | 4   |
+-----+-----+
```

代码

思路：增加两列，使用 lag 函数-把下面的数据往上错位一个，错位 2 个，判断 num 和错位的两列是否相等


```
select

    id

    ,distinct num

from

(

    select

        id

        ,num

        ,lag(num,1) over(partition by id) as lag1

        ,lag(num,2) over(partition by id) as lag2

    from

        log_table

) a

where num=lag1 and lag1=lag2
```

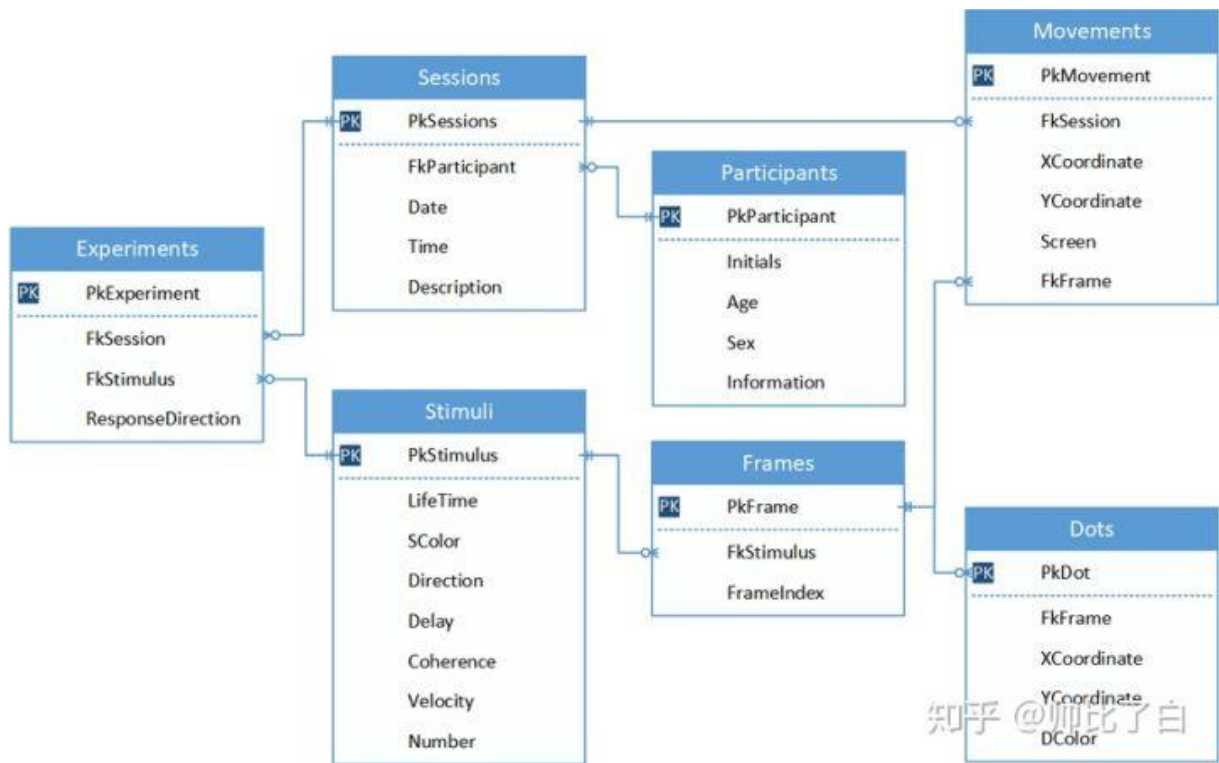
参考

Hive 分析函数 lead、lag 实例应用

sql 中的窗口函数：lead, lag - CodeAntenna

5. SQL 基础 数据库基础【什么是关系型”数据表“,”数据库“】

任何跟数据有关系的岗位、课程、任务，都逃不开数据表、数据库、结构化非结构化数据、关系型数据库非关系型数据库、sql 这些知识。今天咱就缕一缕数据在数据表里，数据表在数据库里，数据库可以是电脑手机，可以是移动硬盘，sql 就是一个钩子，帮你把数据从数据库里拿出来



数据分成结构化数据、非结构化数据，数据库又常分成关系型数据库和非关系型数据库

库

- 结构化数据：就是可以用 excel 存储的，一行行一列列的数据
- 非结构化数据：图片、文字、语音这种没办法用 excel 存储的，一行行一列列的数据

忆臻：结构化数据与非结构化数据的区别 199 赞同 · 9 评论文章

- 关系型数据库：需要预先设计好存储的 structure（数据表之间的关系，unique key, datatype, length）
- 非关系型数据库：可以预先设计好 structure 进行结构化存储，也可以在数据不满足 structure 的情况下先行存储数据，待后来再进行格式化打标签

结构化查询语言(Structured Query Language)简称 SQL，是一种特殊目的的编程语言，是一种数据库查询和程序设计语言，用于存取数据以及查询、更新和管理关系数据库系统

使用 sql 需达到的目标：

1. 会利用 sql 操作关系数据库 mysql 进行查询
2. 数据库的分组、聚合、排序
3. 存储的过程
4. 对于增删改、约束、索引、数据库范式大概了解

推荐一本非常适合新手入门的参考书《SQL 基础教程》，作者是一个日本人，语言简单易懂，配有大量图片，对新手和没有编程基础的人非常友好

mysql 的安装、基本使用操作都写在这篇傻瓜式教学里了帅比了白：SQL 数据分析岗位招聘信息操作练习 53 赞同 · 4 评论文章

想看面试题的可以看这篇面试经验、技巧总结

帅比了白：转专业如何拿到腾讯 offer347 赞同 · 69 评论文章

基础概念速查

数据库=多张表+各表之间的关系

学会数据库需要掌握两点：

1.数据库表的结构 2.各表之间的关系

- 数据库表的结构；数据库每张表长什么样子

表包含带有列名的列，和记录数据的行

每个数据库的表里都会加一个 id 字段用来表示数据的唯一性——主键-没有现实意义

- 各表之间的关系

生活中的联系：你是你爸爸的儿子，你是你儿子的爸爸

表之间通过不同的 id 联系起来

联系就是数据能够相互对应匹配，正式名叫做联接，对应的操作叫 join

- 数据库和 sql 是什么关系

数据库里放着数据，而 sql 使用来操作数据库里数据的语言（工具）

6. 行/列转换

行/列转换在做数据分析时会经常遇到，所谓的行/列转换就是如下图所示两种展示形式的互相转换。

我们假设有 4 款产品 ABCD，分别在三个电商平台天猫、淘宝和京东上进行销售，下表分别以两种形式记录了某个月各产品（Product）在各个平台（Platform）的销售数量（Quantity）。我们为了分析的方便，需要对两种形式进行转换，也就是我们常说的行/列转换。

Product	Platform	Quantity
产品A	天猫	90
产品B	天猫	81
产品C	天猫	79
产品A	淘宝	85
产品B	淘宝	86
产品C	淘宝	92
产品A	京东	87
产品B	京东	98
产品C	京东	93



Product	天猫	淘宝	京东
产品A	90	85	87
产品B	81	86	98
产品C	79	92	93

行转列

假如我们有下表，由 Product、Platform、Quantity 三个字段组成，但是为了方便展示，我们需要将 Platform 中的每一平台展示为一列，也就是行转列。

Product	Platform	Quantity
产品A	天猫	90
产品B	天猫	81
产品C	天猫	79
产品A	淘宝	85
产品B	淘宝	86
产品C	淘宝	92
产品A	京东	87
产品B	京东	98
产品C	京东	93

方法一：PIVOT

在 SQL SERVER 中，提供了专门进行行列转换的函数：PIVOT，PIVOT 的一般语法是：

FROM table_source - 表名称，即数据源

PIVOT (

聚合函数 (value_column) - value_column 要转换为 列值 的列名

FOR pivot_column - pivot_column 指定要转换的列

IN (column_list) - column_list 自定义的目标列名

)

使用 PIVOT 进行行转列的代码如下：

```

SELECT  Product,

        MAX(CASE WHEN Platform = ' 天猫 ' THEN quantity ELSE 0

END) AS ` 天猫 `,

        MAX(CASE WHEN Platform = ' 淘宝 ' THEN quantity ELSE 0

END) AS ` 淘宝 `,

        MAX(CASE WHEN Platform = ' 京东 ' THEN quantity ELSE 0

END) AS ` 京东 `

FROM    TABLE

GROUP BY

        Product

```

列转行

反过来，假设我们有下表 table，各个平台已经作为一列出现，我们需要将各个平台汇总为一列，也就是列转行。

Product	天猫	淘宝	京东
产品A	90	85	87
产品B	81	86	98
产品C	79	92	93

方法一：UNPIVOT

与行转列类似，在 SQL SERVER 中，提供了列转行的函数：UNPIVOT，UNPIVOT 的一般语法是：

```
FROM table_source - 表名称，即数据源

UNPIVOT (

value_column - value_column 要转换为 行值 的列名

FOR pivot_column - pivot_column 指定要转换为指定的列

IN(column_list ) - column_list 目标列名

)
```

使用 UNPIVOT 进行列转行的代码如下：

```

SELECT  *

FROM    TABLE UNPIVOT (quantity FOR Platform IN (" 天猫 ", " 淘
宝 ", " 京东 "))

```

通过 UNPIVOT 可得到如下结果：

Product	Platform	Quantity
产品A	天猫	90
产品B	天猫	81
产品C	天猫	79
产品A	淘宝	85
产品B	淘宝	86
产品C	淘宝	92
产品A	京东	87
产品B	京东	98
产品C	京东	93

方法二：聚合函数+UNION

同理，Oracle，MySQL 也是不支持 UNPIVOT 函数的，对于这种情况我们可以通过聚合函数+UNION 的方式将数据组合起来。

7. 一行/多行转换

在数据分析中，还有一个非常常见的场景，就是需要将一行数据拓展为多行数据，或者将多行数据汇总为一行数据，这种一行/多行的数据转换问题也是面试中频繁考查的知识点之一，举例如下图所示。

Product	Supplier
产品A	Sup_1
产品A	Sup_2
产品A	Sup_3
产品B	Sup_1
产品B	Sup_2
产品B	Sup_4
产品C	Sup_2
产品C	Sup_3
产品C	Sup_4



Product	Supplier
产品A	Sup_1, Sup_2, Sup_3
产品B	Sup_1, Sup_2, Sup_4
产品C	Sup_2, Sup_3, Sup_4

一行转多行

原数据如下：table 表中有两个字段，产品（Product）和供应商（Supplier），每个产品有多个供应商，各个供应商以逗号分割汇总为一行。现在需要将各个供应商分别展开为一行。

Product	Supplier
产品A	Sup_1, Sup_2, Sup_3
产品B	Sup_1, Sup_2, Sup_4
产品C	Sup_2, Sup_3, Sup_4

针对这种情况，Hive 中提供了一个很高效的方法：lateral view 和 explode 等函数一起使用，能将一行数据拆分成多行数据，在此基础上可以对拆分的数据进行聚合，使用的时候需要注意：

1、explode 接受一个数组（或一个 map）作为输入，并将数组元素（map）作为单独的行输出。UDTF 可以在 SELECT 表达式列表中使用，也可以作为 LATERAL VIEW 的一部分使用。

2、LATERAL VIEW 是配合 explode（或者其他的 UDTF），把单行数据拆解成多行后的数据结果集。

3、LATERAL VIEW explode 要放置于 from 之后，where 之前

```
SELECT DISTINCT Product,Supplier
```

```
FROM table
```

```
LATERAL VIEW explode (split(Supplier,' ')) as t - t 为新表别名
```

拆成后结果：

Product	Supplier
产品A	Sup_1
产品A	Sup_2
产品A	Sup_3
产品B	Sup_1
产品B	Sup_2
产品B	Sup_4
产品C	Sup_2
产品C	Sup_3
产品C	Sup_4

多行转一行

反过来，我们想把多行再变回一行，需要怎么处理呢？我们可以使用：`collect_set` + `concat_ws` 来实现。代码如下：

```
SELECT Product,

       concat_ws(' ', COLLECT_SET(Supplier)) Product_Supplier

FROM   TABLE

GROUP BY

       Product
```

最终还原为一行数据如下：

Product	Supplier
产品A	Sup_1, Sup_2, Sup_3
产品B	Sup_1, Sup_2, Sup_4
产品C	Sup_2, Sup_3, Sup_4

注意：上述用的到的 `collect_set` 函数，有两个作用，第一个是去重，去除 `group by` 后的重复元素，

第二个是形成一个集合，将 `group by` 后属于同一组的集合起来成为一个集合。与 `concat_ws` 结合使用就是将这些元素以逗号分隔的形式形成字符串。

8. 计算连续活跃

连续活跃或者登录的计算是数据分析业务场景中非常经典和常见的问题，而且涉及到的解题思路比较巧妙，所以也是面试笔试中非常喜欢考查的知识点，通过下面的笔试题的详细讲解，希望小伙伴们可以举一反三，把计算连续这类问题吃透。

记录用户活跃时间信息的表 `user_active` 如下：

userid	active_time
10001	2021-01-01 07:12:00
10001	2021-01-01 15:23:00
10001	2021-01-02 09:46:00
10001	2021-01-02 16:14:00
10001	2021-01-03 07:48:00
10002	2021-01-02 07:26:00
10002	2021-01-02 18:53:00
10002	2021-01-03 09:27:00
10002	2021-01-04 19:29:00
10002	2021-01-04 10:05:00

需求：计算连续活跃 2 天及以上的用户。

第 1 步：用户活跃日期去重

因为一个用户同一天可能活跃多次，我们只需要知道用户在某天是否活跃即可，所以需要对用户 id+活跃日期去重。

```
SELECT  DISTINCT CAST(active_time AS date) active_day,  
        userid  
  
FROM    user_active;
```

查询结果:

userid	active_day
10001	2021-01-01
10001	2021-01-02
10001	2021-01-03
10002	2021-01-02
10002	2021-01-03
10002	2021-01-04

第 2 步：窗口函数按日期排序

有了第 1 步去重后的结果，我们可以用 row_number() 函数对其进行开窗，以 userid 分组，日期升序排序，得到每个用户按照日期升序的排名。

```

SELECT  *,

        row_number() OVER(

            PARTITION BY

                userid

            ORDER BY

                active_day

        ) AS rn

FROM    (

        SELECT  DISTINCT CAST(active_time AS date)

        active_day,

                userid

        FROM    user_active;

    ) a;

```

查询结果：

userid	active_day	rn
10001	2021-01-01	1
10001	2021-01-02	2
10001	2021-01-03	3
10002	2021-01-02	1
10002	2021-01-03	2
10002	2021-01-04	3

看到这里，有些小伙伴可能就比较疑惑了，为什么要在这一步对日期进行排序呢？这也是计算连续活跃、达标逻辑比较 tricky 的地方，因为我们用 row_number 排序之后的名次是连续的，那么如果恰巧日期也是连续的，日期和排序差值不就是一个固定的值了吗？所有连续的日期是不是都被这个固定的值圈定了？

第 3 步：确定连续日期的分组

按照上面所说的逻辑，我们使用 date_sub 函数将当前日期 active_day 和当前的排序值 rn 相减，得到一个新的列，命名为"day_group"

```

SELECT  *,
        DATE_SUB(active_day, rn) AS day_group
FROM    (
        SELECT  *,
                row_number() OVER(
                    PARTITION BY
                        userid
                    ORDER BY
                        active_day
                ) AS rn
        FROM    (
                SELECT  DISTINCT CAST(active_time AS
date) active_day,
                        userid
                FROM    user_active;
            ) AS a
        ) b;

```

查询结果：

userid	active_day	rn	day_group
10001	2021-01-01	1	2020-12-31
10001	2021-01-02	2	2020-12-31
10001	2021-01-03	3	2020-12-31
10002	2021-01-02	1	2021-01-01
10002	2021-01-03	2	2021-01-01
10002	2021-01-04	3	2021-01-01

第 4 步：根据分组列计算连续天数

最后一步，我们直接根据第 3 步中获取到的分组列，根据 userid 和分组列进行分组求 count 即可。如果是要求连续活跃 2 天以上，我们直接对聚合后的 count 用 having 进行过滤即可。

```

SELECT  userid,

        day_group,

        count(*) AS continous_days

FROM    (

        SELECT  *,

                DATE_SUB(active_day, rn) day_group

        FROM    (

                SELECT,

                        row_number() OVER(

                                PARTITION BY

                                        userid

                                ORDER BY

                                        active_day

                        ) AS rn

                FROM    (

                        SELECT  DISTINCT

                                CAST(active_time AS date) active_day,

                                        userid

                                FROM    user_active

                        ) a

                ) b

        ) c

GROUP BY

        userid,

        day_group

HAVING  count() >= 2;

```


查询结果:

userid	day_group	continous_days
10001	2020-12-31	3
10002	2021-01-01	3

到这里结果就出来了，userid 为 10001 和 10002 的用户都连续活跃了 3 天，也就是符合我们条件的用户。

9. 计算累计指标

求累计也是数据分析实际业务中非常常见的场景，截止到每天的累计销售额、截止到每天的注册用户数，可以通过这些指标来看 KPI 完成情况。所以累计的计算也是笔试中经常出现的题目，下面通过一个案例讲解如何计算累计，抛砖引玉，希望大家能举一反三。

记录每个商品每天销售量的表 sale_info 如下图：

product_id	sale_date	quantity
A0001	2021-01-21	5
A0002	2021-01-23	6
A0003	2021-01-22	8
A0004	2021-01-20	3
A0001	2021-01-23	6
A0001	2021-02-21	8
A0002	2021-01-23	6
A0001	2021-02-22	4

需求：使用 SQL 统计出每个商品截至每个月的累积销量。

第 1 步：计算每个商品每个月的销量

为了方便后面的讲解，我们把这一步操作的结果存在临时表 tmp 中，第 1 步我们先计算每个商品每个月的销量。

```
CREATE TABLE tmp AS

SELECT product_id,

       substring(sale_date, 1, 7) sale_month,

       SUM(quantity) month_quantity

FROM   sale_info

GROUP BY

       product_id,

       substring(sale_date, 1, 7);
```

查询结果：

product_id	sale_month	month_quantity
A0001	2021-01	11
A0001	2021-02	12
A0002	2021-01	12
A0003	2021-01	8
A0004	2021-01	3

第 2 步：按月累计计算销量

到这一步，我们用 sum 开窗函数，对 userid 进行分组、按照月份进行排序即可完成累计销量的计算。

```
SELECT  product_id,

        sale_month,

        month_quantity,

        SUM(month_quantity) OVER(

            PARTITION BY

                product_id

            ORDER BY

                sale_month

        ) month_all

FROM    tmp;
```

查询结果：

product_id	sale_month	month_quantity	Month_all
A0001	2021-01	11	11
A0001	2021-02	12	23
A0002	2021-01	12	12
A0003	2021-01	8	8
A0004	2021-01	3	3

完整的 SQL 代码如下：

```

SELECT  product_id,

        sale_month,

        month_quantity,

        SUM(month_quantity) OVER(

            PARTITION BY

                product_id

            ORDER BY

                sale_month

        ) month_all

FROM    (

        SELECT  product_id,

                substring(sale_date, 1, 7) sale_month,

                SUM(quantity) month_quantity

        FROM    sale_info

        GROUP BY

                product_id,

                substring(sale_date, 1, 7)

        ) t;

```

10. 去除最高最低的平均

salary_info 表记录了每个员工的月工资以及所属的部门，包含 EmpId(员工 ID)、Department(部门名称)、Salary(月工资)。

Salary_info		
Empld	Department	Salary
10001	市场部	9000
10002	市场部	8000
10003	市场部	7900
10004	市场部	8800
10005	产品研发部	8500
10006	产品研发部	8600
10007	产品研发部	9200
10008	产品研发部	8300
10009	运营部	8700
10010	运营部	9800
10011	运营部	9300
10012	运营部	8600

需求：计算除去部门最高工资，和最低工资的平均工资（字节跳动面试）。

因为要在每个部门内计算平均工资，但是要去除该部门的最高工资和最低工资，所以我们可以考虑使用窗口函数，按照部门进行分组，但是如何将部门的最高工资和最低工资去除呢？我们可以对分组后的数据按照工资分别顺序和逆序进行排序，排名第一的就分别是该部门最低和最高工资，将这 2 个排名第一的记录去除就是我们要计算的平均工资。

核心是使用窗口函数 RANK 分别对工资 salary 进行升序和降序排列，就获得了该分组内最低和最高的工资，过滤掉这 2 条记录再对工资 salary 进行平均 avg 即可。

完整 SQL 代码如下：

```

SELECT  a.department ,  avg(a.salary)

FROM    (

        SELECT  *,

                RANK() OVER (

                        PARTITION BY

                                department

                        ORDER BY

                                salary

                ) rank_asc,

                RANK() OVER (

                        PARTITION BY

                                department

                        ORDER BY

                                salary DESC

                ) rank_desc

        FROM    emp

    ) a

GROUP BY

        a.department

WHERE   a.rank_asc > 1

AND     a.rank_desc > 1

```

11. 计算留存率

留存率是用户运营中非常关键的指标，也是面试中非常喜欢考查的题型，给出如下某个 App 的活跃日志表 active_log，记录了某个用户 userid 每天 active_day 在 App 上登录次数 login_cnt 和登录时长 duration（分钟），如下图。

userid	active_day	login_cnt	duration
10001	2020-05-01	2	1
10001	2021-05-02	2	3
10001	2021-05-03	1	3
10002	2021-05-01	3	2
10002	2021-05-02	2	2
10003	2021-05-01	3	4
10004	2021-05-01	5	4
10004	2021-05-03	1	2
10005	2021-05-01	2	2
.....			CSDN@数据分析星球

需求：计算该 App 的次日留存率和 7 日留存率。

第 1 步：自连接进行日期连接

计算时间间隔类问题，我们需要使用自联结，得到如下形式的格式对于计算留存率十分关键，我们把临时表放到 tmp_1 表中。

```

CREATE TABLE tmp_1 AS

SELECT DISTINCT a.userid,

               a.active_day AS a_day,

               b.active_day AS b_day

FROM   active_log AS a

LEFT JOIN

       active_log AS b

ON     a.userid = b.userid;

```

查询结果：

userid	a_day	b_day
10001	2020-05-01	2020-05-01
10001	2021-05-01	2021-05-02
10001	2021-05-01	2021-05-03
10001	2021-05-02	2021-05-01
10001	2021-05-02	2021-05-02
10001	2021-05-02	2021-05-03
10001	2021-05-03	2021-05-01
10001	2021-05-03	2021-05-02
10001	2021-05-03	2021-05-03
*****	*****	**** @数据分析星球

第 2 步：计算时间间隔

自连接后，计算时间间隔 day_diff

```

select *,

date_sub(b_day,a_day) as day_diff

from tmp_1;

```


查询结果：

userid	a_day	b_day	daydiff
10001	2020-05-01	2020-05-01	0
10001	2021-05-01	2021-05-02	1
10001	2021-05-01	2021-05-03	2
10001	2021-05-02	2021-05-01	-1
10001	2021-05-02	2021-05-02	0
10001	2021-05-02	2021-05-03	1
10001	2021-05-03	2021-05-01	-2
10001	2021-05-03	2021-05-02	-1
10001	2021-05-03	2021-05-03	0
.....			CSDN @数据分析星球

第3步：计算次日、7日留存数

使用 case when 计算次日留存数、7日留存数

```

SELECT  d.a_day,

        count(DISTINCT CASE WHEN d.day_diff = 1 THEN d.userid

ELSE NULL END) AS `次日留存数`,

        count(DISTINCT CASE WHEN d.day_diff = 7 THEN d.userid

ELSE NULL END) AS `7 日留存数`

FROM    (

        SELECT  *,

                DATE_SUB(b_day, a_day) AS day_diff

        FROM    tmp_1

    ) d

GROUP BY

        d.a_day;

```

查询结果：

a_day	次日留存数	7日留存数
2020-05-01	2	1
2021-05-02	1	0
2021-05-03	1	0
2021-05-04	2	1
2021-05-05	1	1
2021-05-06	2	1
2021-05-07	2	2
2021-05-08	0	0
2021-05-09	1	1
.....		CSDN @数据分析星球

第 4 步：完整 SQL 代码

计算每日的活跃用户数，次日、7 日留存率的完整的代码如下：

```

SELECT  d.a_day,

        count(DISTINCT d.userid) AS `活跃用户数`,

        count(DISTINCT CASE WHEN d.day_diff = 1 THEN d.userid

ELSE NULL END) AS `次日留存数`,

        count(DISTINCT CASE WHEN day_diff = 1 THEN d.userid ELSE

NULL END) / count(DISTINCT d.userid) AS `次日留存率`,

        count(DISTINCT CASE WHEN d.day_diff = 3 THEN d.userid

ELSE NULL END) AS `3 日留存数`,

        count(DISTINCT CASE WHEN day_diff = 3 THEN d.userid ELSE

NULL END) / count(DISTINCT d.userid) AS `3 日留存率`,

        count(DISTINCT CASE WHEN d.day_diff = 7 THEN d.userid

ELSE NULL END) AS `7 日留存数`,

        count(DISTINCT CASE WHEN day_diff = 7 THEN d.userid ELSE

NULL END) / count(DISTINCT d.userid) AS `7 日留存率`,

FROM    (

        SELECT  *,

                DATE_SUB(b_day, a_day) AS day_diff

        FROM    tmp_1

    ) d

GROUP BY

        d.a_day;

```

完整的结果如下：

a_day	活跃用户数	次日留存数	次日留存率	7日留存数	7日留存率
2020-05-01	5	2	40%	1	20%
2021-05-02	2	1	50%	0	0%
2021-05-03	2	1	50%	0	0%
2021-05-04	3	2	66%	1	33%
2021-05-05	2	1	50%	1	50%
2021-05-06	3	2	66%	1	33%
2021-05-07	5	2	40%	2	40%
2021-05-08	1	0	0%	0	0%
2021-05-09	2	1	50%	1	50%
.....				CSDN @数据分析星球	

12. 计算分类 TopN

另外一个非常常见的场景是计算前几名 TopN，比如每个商品类别下最受欢迎的 TopN 产品，每个片区销售额最高的 TopN 店铺等。

假设有一个网店，上线了 100 多个商品，每个顾客浏览任何一个商品时都会产生一条浏览记录，浏览记录存储的表名为 product_view，访客的用户 id 为 user_id，浏览的商品名称是 product_id。

user_id	Product_id
A0001	product_a
A0002	product_b
A0001	product_b
A0001	product_a
A0003	product_c
A0004	product_b
A0001	product_a
A0002	product_c
A0005	product_b
A0004	product_b
A0006	product_c
A0002	product_c
A0001	product_b
A0002	product_a
A0002	product_a
A0003	product_a
A0005	product_a
A0005	product_a
A0005	product_a

需求：每个商品浏览次数 top3 的用户信息，输出商品 id、用户 id、浏览次数。

第 1 步：计算每个商品被每个用户浏览的次数

因为我们最终需要获取每个商品浏览量 top3 的用户信息，所以第一步，我们要先把每个商品下每个用户的浏览次数计算出来，放在临时表 t1 中。

```

CREATE TABLE t1 AS

SELECT product_id,

       user_id,

       count(*) cnt

FROM   product_view

GROUP BY

       product_id,

       user_id;

```

查询结果：

product_id	user_id	cnt
product_a	A0001	3
product_a	A0002	2
product_a	A0003	1
product_a	A0005	3
product_b	A0001	2
product_b	A0002	1
product_b	A0004	2
product_b	A0005	1
product_c	A0002	2
product_c	A0003	1
product_c	A0006	1

第 2 步：每个商品被浏览次数排名

有了上一步每个商品下的各用户的浏览量，我们想获取 Top3 浏览量的用户信息，毫无疑问，

我们需要使用到排序的开窗函数，但是排序窗口函数有：ROW_NUMBER，RANK 或者 DENSE_RANK，我们应该用哪个呢？三个函数的区别如下：

ROW_NUMBER 从 1 开始，按照 ORDER BY 的顺序，值相等时排名不出现并列；

RANK 与 ROW_NUMBER 类似，只是值相等时，排名会并列，并会在名次中跳过并列排名继续排名；

DENSE_RANK 与 ROW_NUMBER 类似，只是值相等时，排名会并列，并会在名次中紧接着并列的排名继续排名。

ROW_NUMBER 对于相同数据的排名不是一样的，如果我们取 Top3，出现了相同访问次数的数据，那我们都需要保留下来，所以这里我们使用 RANK 函数。

```
CREATE TABLE t2 AS

SELECT product_id,

       user_id,

       cnt,

       RANK() OVER (

           PARTITION BY

               product_id

           ORDER BY

               cnt

       ) rn

FROM t1;
```

查询结果：

product_id	user_id	cnt	rn
product_a	A0003	1	1
product_a	A0002	2	2
product_a	A0005	3	3
product_a	A0001	3	3
product_b	A0005	1	1
product_b	A0002	1	1
product_b	A0004	2	3
product_b	A0001	2	3
product_c	A0006	1	1
product_c	A0003	1	1
product_c	A0002	2	3

CSDN @ 数据分析师星球

第 3 步：计算每个商品浏览前 3 的用户

有了第 2 步的结果，我们想要取每个商品浏览前三的用户信息就很简单了。

```
select product_id,user_id,cnt
from t2
where rn<=3;
```

查询结果:

product_id	user_id	cnt
product_a	A0003	1
product_a	A0002	2
product_a	A0005	3
product_a	A0001	3
product_b	A0005	1
product_b	A0002	1
product_b	A0004	2
product_b	A0001	2
product_c	A0006	1
product_c	A0003	1
product_c	A0002	2

完整的 SQL 代码如下：

```

SELECT  product_id,

        user_id,

        cnt

FROM    (

        SELECT  product_id,

                user_id,

                cnt,

                RANK() OVER(

                        PARTITION BY

                                product_id

                        ORDER BY

                                cnt

                ) rn

        FROM    (

                SELECT  product_id,

                        user_id,

                        count(*) cnt

                FROM    product_view

                GROUP BY

                        product_id,

                        user_id

                ) t1

        ) t2

WHERE   rn <= 3;

```

13. 计算连续得分

现有一张表 score_info 记录了一场篮球比赛中各个球员的得分记录，即某个球员 userid 得分了，就记录该球员的得分时间 score_time 和得分 score。如下表所示。

user_id	score_time	score
A0001	t1	2
A0001	t2	2
A0002	t3	3
A0001	t4	2
A0001	t5	2
A0001	t6	2
A0003	t7	3
A0003	t8	2

CSDN @数据分析星球

需求：计算连续 3 次得分的用户数，而且中间不能有别的球员得分。

因为要计算连续得分，这个时候我们的第一反应就应该就是用窗口函数的排序 RANK，但是需要计算每个球员连续 3 次得分，我们思考一下，如果一个球员连续三次得分，那么整体得分记录的次序和自己得分记录的次序是不是保持同步的增长，两者之间差一个恒定的值？如果这个恒定的值连续出现 3 次及以上，那么这个球员就应该是我们要统计的球员。

那么这个题目的核心问题就变成了计算：1、整体得分记录的次序；2、每个球员得分记录的次序。分别计算如下，分别用 rn_all 和 rn_user 表示。

```
rank() over(order by score_time) as rn_all
```

```
rank() over(partition by user_id order by score_time) as rn_user
```

得到的 rn_all 和 rn_user，以及 diff = rn_all - rn_user 如下所示：

user_id	score_time	score	rn_all	rn_user	rn_all-rn_user
A0001	t1	2	1	1	0
A0001	t2	2	2	2	0
A0002	t3	3	3	1	2
A0001	t4	2	4	3	1
A0001	t5	2	5	4	1
A0001	t6	2	6	5	1
A0003	t7	3	7	1	6
A0003	t8	2	8	2	6

这时我们发现只需要对 diff 进行分组计数大于 3 个,就是连续点击大于三且中间没有其他人点击的用户。

完整的 SQL 代码如下：

```

SELECT  user_id,

        diff,

        count(1)

FROM    (

        SELECT  *,

                rn_all - rn_user AS diff

        FROM    (

                SELECT  *,

                        row_number() OVER(

                                ORDER BY

                                        score_time

                                ) AS rn_all row_number() OVER(

                                PARTITION BY

                                        user_id

                                ORDER BY

                                        score_time

                                ) AS rn_user

                FROM    socre_info

                ) a

        ) b

GROUP BY

        user_id,

        diff

HAVING  count(1) >= 3;

```

14. 用户行为分析

业务背景

某购物 APP 最近上线了一个新功能，用户签到后可以跳转到大转盘抽奖，抽奖获得的奖金可以抵消购物的费用，以此来培养用户使用 app 的习惯。

数据表介绍

现有一张用户行为表 user_log，主要字段如下，记录了用户在 app 上的所有行为日志，即何人 user_id 在何时 event_time 进行了什么操作 event_id。

user_log		
user_id	event_time	event_id
10001	2021-01-01 07:12:05	login
10001	2021-01-01 07:13:47	register
10001	2021-01-02 09:46:26	login
10001	2021-01-02 09:47:15	register
10001	2021-01-02 09:47:25	gift
10001	2021-01-02 09:52:01	logout
10002	2021-01-03 18:53:02	login
10002	2021-01-03 18:54:10	view
10002	2021-01-03 18:56:34	cart
10002	2021-01-03 18:58:24	check

需求：

- 1、计算该 app 每天的访客数以及每天人均行为次数。
- 2、统计每天签到之后并进行抽奖的用户数，注意签到和抽奖行为必须相邻（签到和抽奖行为对应的 event_id 分别为 'register'，'gift'）。

思路：

第 1 问比较简单，计算 app 每天的访客数，因为用户登录访问 app 就会在表中产生对应的行为日志，所以每天的访客数只需要按天对用户数去重即可，每天人均行为次数的计算，因为一

次行为就会产生一条记录，所以，人均行为次数就是所有的记录计数，除以总的访客数。

代码如下：

```
select cast(event_time as date) as day,  
count(distinct user_id) as active_cnt,  
count( event_id)/count(distinct user_id) as avg_opr_cnt  
from user_log  
group by cast(event_time as date);
```

第2问升级了难度，虽然也是统计用户数，但是添加了限制：签到之后要大转盘抽奖，两个行为一前一后必须相邻才可以。这个时候我们可以用窗口函数的位移函数 `lead ( ) over ( )` 实现，`lead` 可以取当前记录的下一条记录，如果我们对每个用户 `user_id` 分组，按照行为时间 `event_time` 升序排列，就可以得到一个用户的连续的行为记录，再用 `lead ( )` 就可以得到下一条记录，从而在当前记录中得到下一条记录，对两个连续行为进行筛选，就可以计算满足这个条件的用户数。

代码如下：


```

SELECT  a.day,

        count(DISTINCT user_id)

FROM    (

        SELECT  user_id,

                CAST(event_time AS date) AS DAY,

                event_id,

                LEAD(event_id, 1) OVER(

                        PARTITION BY

                                user_id

                        ORDER BY

                                event_time

                ) AS next_event_id

        FROM    user_log

        ) a

WHERE   event_id = ' register '

AND     next_event_id = ' gift '

GROUP BY

        a.day;

```

15. 活动效果数据分析

业务背景

为了提高某店铺的营业额，最近运营部门上线了多个运营活动，用户参与活动后可以领取不定额的优惠券，以此鼓励用户下单。但每个用户限参与一个活动，不可同时参与多个活动。

数据表介绍

现有一张订单表 orders 和活动报名表 act_join,分别记录了用户的下单明细和用户报名的活动

明细。具体字段如下：

订单表 orders，大概字段有（ user_id ‘用户编号’ ,order_id ‘订单编号’ ， order_sales ‘订单金额’ ， order_time ‘下单时间’ ）。

order			
user_id	order_id	order_sales	order_time
10001	ord_13361	120	2021-07-01 07:12:05
10002	ord_13247	100	2021-07-03 07:13:47
10003	ord_10924	150	2021-07-05 09:46:26
10005	ord_13071	250	2021-07-04 09:47:15
10006	ord_14533	450	2021-07-08 09:47:25
10007	ord_11134	140	2021-07-12 09:52:01
10008	ord_14305	180	2021-07-09 18:53:02
10009	ord_13055	220	2021-07-14 18:54:10
10011	ord_10460	190	2021-07-10 18:56:34
10012	ord_11959	210	2021-07-13 18:58:24

活动报名表 act_join，大概字段有（ act_id ‘活动编号’ ，user_id ‘报名用户’ ,join_time ‘报名时间’ ）

act_join		
act_id	user_id	join_time
act_01	10001	2021-06-30 07:12:05
act_01	10002	2021-07-02 07:13:47
act_01	10003	2021-07-01 09:46:26
act_01	10005	2021-07-02 09:47:15
act_02	10006	2021-07-04 09:47:25
act_02	10007	2021-07-01 09:52:01
act_02	10008	2021-07-03 18:53:02
act_03	10009	2021-07-04 18:54:10
act_03	10011	2021-07-05 18:56:34
act_03	10012	2021-07-13 18:58:24

需求：

统计每个活动报名的所有用户在报名后产生的总订单金额，总订单数。（每个用户限报一个活动,且默认用户报名后产生的订单均为参加活动的订单）。

统计每个活动从开始后到当天（数据统计日）平均每天产生的订单数，活动开始时间定义为最早有用户报名的时间。

思路：

第1问计算总订单金额和总订单数，这两个指标都比较简单 `sum(order_sales)`、`count(order_id)` 就可以，但是关键在于限定条件，是每个活动报名后的用户的汇总，必须是报名了某个活动，且必须在活动开始后的数据统计。可以通过订单表 `orders` 和报名表 `act_join` 连接，限定订单时间大于等于活动的开始时间即可。

代码如下：

```

SELECT  t2.act_id,

        count(t1.order_time) AS order_cnt,

        SUM(order_sales) AS order_sales_sum

FROM    (

        SELECT  user_id,

                order_id,

                order_sales,

                order_time

        FROM    orders

        ) t1

INNER JOIN

        (

        SELECT  user_id,

                act_id,

                join_time

        FROM    act_join

        ) t2

ON      t1.user_id = t2.user_id

WHERE   t1.order_time >= t2.join_time

GROUP BY

        t2.act_id;

```

第 2 问与第 1 问有相似之处，同样是用户报名后的下单，只是多了一些限定条件：同时要满足要小于等于计算日期当天，也就是程序运行的系统时间 now()，在此基础上，计算整体的订单

数，除以活动进行的天数，就是该活动每天的平均下单数。

代码如下：

```

SELECT  t1.act_id,

        count(order_id) / datediff(NOW(), MIN(t1.begin_time)) -

总店单数 / 活动天数

FROM      (

        SELECT  act_id,

                user_id,

                join_time,

                MIN(join_time) OVER(

                        PARTITION BY

                                act_id

                                ) AS begin_time --当前活动的开始时间

        FROM      act_join

        ) t1

INNER JOIN

        (

        SELECT  user_id,

                order_id,

                order_time

        FROM      orders

        ) t2

ON      t1.user_id = t2.user_id

WHERE   t1.join_time BETWEEN t1.begin_time AND NOW() --活动开始至今

的数据

AND      t2.order_time >= t1.join_time --活动开始后的下单

GROUP BY

        t1.act_id;

```

说明：这里使用了窗口函数，计算了每个活动的开始时间，然后 join 订单表，通过 where 条件将上面的限定条件满足。当然这只是一种解题思路，还有很多其他的解法，大家可以尝试。

16. 用户网购行为分析

1) 需求：

分析有订单事务表：orders

加购事务表：addcart

请用 SQL 代码分析用户对商品的行为特征，包括：已购买，购买未加购，加购未购买，加购且购买。

2) 解题思路：

1.需要得到 user_id 和 item_id 的购买和加购的组合情况，使用这两个主键进行关联。

2.根据是否可以关联来判断是否购买和是否加购。

3.使用 full join 外连接，MySQL 不支持 full join，可以使用左连接和右连接后使用 union 即可。

代码如下：

```

SELECT  a.user_id,

        a.item_id,

        1 AS `已购买`,

        CASE WHEN b.item_id IS NULL THEN 1 ELSE 0 END AS `购买未
加购`,

        0 AS `加购未购买`,

        CASE WHEN a.item_id = b.item_id THEN 1 ELSE 0 END AS `加
购且购买`,

FROM    orders a

LEFT JOIN

        addcart b

ON      a.user_id = b.user_id

AND     a.item_id = b.item_id

UNION

SELECT  a.user_id,

        a.item_id,

        CASE WHEN b.item_id IS NOT NULL THEN 1 ELSE 0 END AS `已
购买`,

        0 AS `购买未加购`,

        CASE WHEN b.item_id IS NULL THEN 1 ELSE 0 END AS `加购未
购买`,

        CASE WHEN a.item_id = b.item_id THEN 1 ELSE 0 END AS `加
购且购买`,

FROM    addcart a

LEFT JOIN

```


17. 销售额下降了如何分析？

1) 需求

这是一道非常经典的业务题。

如果我们发现，某店铺 2020 年 11 月的销售额同比去年 11 月下降了 60%，如果你是这块业务的数据分析师，你会如何分析？请写出你的分析思路框架。

2) 解题思路：

1、先确认数据来源是否可靠，指标口径是否一致，很多时候，大家只是道听途说，讲的不是同一个事情，埋头就开始分析，反而浪费了时间精力，需要首先确认问题是否真实存在。

2、如果数据来源可靠，指标口径一致，先看外部市场环境，是否是整体市场大盘在下降，受到大盘的影响。

3、如果不是外部因素，看内部问题，对内部因素进行指标成分拆解，可以按照以下的方式进行拆解。

4、具体可能的拆解方式如下：

销售额 = 访客数 * 下单率 * 客单价 - 退货金额

访客数 = 新客户 + 老客户，新客户 = 渠道的流量 * 转化率

下单率 = 页面访问量 * 转化率

订单取消率 = 订单取消数 / 订单总数

5、针对指标的变化进行提建议，定位到具体的原因后，针对性的给出改进策略方案，如开展促销活动、商品。

18. 玩转数据分析中常见的 SQL 函数

1、写在前面

SQL 作为数据分析师必备技能之一，无论是初级分析师还是高级分析师，SQL 已经是各大公司招聘条件里的必选项，为什么 SQL 对于数据分析师来说如此重要呢？在回答这个问题之前，我们先搞懂以下几个问题。

第一个问题，SQL 是啥？

SQL 是 Structured Query Language 的缩写，意思是结构化查询语言，是一种在数据库管理系统 (Relational Database Management System, RDBMS) 中查询数据，或通过 RDBMS

对数据库中的数据进行更改的语言。

看不懂。。能不能说人话？好嘞，SQL 就是一种对数据库中的数据表或者数据进行增、删、改、查等操作的语言。

什么是数据库？“数据库是“按照数据结构来组织、存储和管理数据的仓库”。是一个长期存储在计算机内的、有组织的、可共享的、统一管理的大量数据的集合。说人话就是按照一定的组织结构存储数据的仓库。我们常见的 Oracle，MySQL，SQL Server 都是数据库，只是有一些是商业的数据库，一些是开源免费的而已。

第二个问题，数据分析为啥要学 SQL？

其实理解了数据库和 SQL 的关系之后，这个问题就是句废话了。巧妇难为无米之炊，数据分析的第一步肯定是要有数据，数据哪里来？肯定要从数据库中取出来，SQL 就是这样一个方便、普适的取数工具，因为几乎所有的数据库的 SQL 语法都是相似的，甚至现在我们做大数据分析用到的 Hive SQL，其语法 99%也是和 SQL 一样的，所以学会 SQL 基本上就掌握了所有数据库的取数方法。

第三个问题，到底怎么学习 SQL？

你以为 SQL 只是一个简单的取数工具，把数据从数据库中拉出来就完事了？

1、SQL 不仅能取数据，还提供了丰富的函数，可以做数据的清洗、转换等数据处理，而且 SQL 还能像 Excel 的透视表那样，可以方便地在不同的维度上对数据进行求和、计数、去重计数、求平均等操作，进而对数据进行分析，而这一切，也只需要短短的几行 SQL 代码就能实现。

2、如果数据很多很复杂，像 Excel 那样存放在不同的 Sheet 里，要汇总在一起进行分析，要怎么办呢？SQL 的强大之处就在于可以非常方便地将不同的数据按照一定的关联连接起来，这个关联可以是内连接 inner join（找两个表的交集）、左连接 left join（交集并且左表所有）、右连接（right join 交集并且右表所有）、全连接 outer join（找两个表的并集），可以通过各种不同的关联条件可以实现各种不同的数据连接，最终对连接后的数据进行分析。

3、通过以上两点来看，SQL 好像和 Excel 功能上没啥区别啊？Excel 也能做数据清洗，透视表也能做求和、计数等聚合操作，Excel 的 Power Pivot 也能实现多个表之间的连接。实际上，SQL 除了以上这些功能之外，还提供了一个非常强大的功能：窗口函数，窗口函数有什么用呢？如果我们要计算每个人在特定分组下的排名、每月销售额的同比、环比、截至每天的累计销售额，这些数据分析中经常遇到的、且基础的 SQL 语句无法很好解决的问题，窗口函数就显示出它的威力了。所以窗口函数也是判断你是 SQL 基础玩家和高阶玩家的重要标准，也是数据分析面试中最喜欢考查的内容之一。

那么想入行数据分析的同学来说，怎么快速高效地掌握 SQL 这个数据分析的利器呢？根据前面的介绍，提升 SQL 水平可以按照这样的学习路径：

1、SQL 基础语法：首先熟悉 SQL 的基础语法，对于数据分析而言，重点掌握数据查询 SELECT，包括：如何使用 WHERE 进行数据筛选，熟练使用算数运算符（ $\pm$ */）、逻辑运算符（AND /OR/NOT）进行字段计算和条件过滤，使用 SUM、COUNT、AVG 等聚合函数结合 GROUP BY 进行不同维度下的汇总分析，如何用 HAVING 子句对聚合的结果进行过滤，并使用 ORDER BY 对最终的查询结果进行排序。这一部分最重要的一点是：要明确 SQL 语句的执行顺序与书写顺序的差异，这一点对于了解 SQL 的执行过程很有帮助。作为 SQL 系列文章的第一篇内容，我们会在本文中重点讲解。

2、SQL 常用函数：在掌握了 SQL 基础语法的基础上，下面就要熟练掌握一些数据分析中常用的函数，包括但不限于：日期类函数、字符串类函数，数值运算类函数等。熟练使用这些函数可以帮助我们高效地做数据的清洗、转换等数据处理工作。这部分内容我们会在 SQL 系列文章的第二篇重点讲解。

3、SQL 子查询与表连接：在之前的基础上，我们还要熟练掌握子查询和多个表之间的连接。数据分析工作中，往往需要对多张有关联的表进行分析，对于简单的查询，我们可以使用嵌套的子查询解决。但如果涉及的表很多，且表之间的关联关系比较复杂，我们就需要使用表连接，按照一定的关联关系将各个表连接在一起，常见的连接类型有内连接：INNER JOIN、左连接：LEFT JOIN、右连接：RIGHT JOIN、全连接：FULL JOIN。这部分内容我们会在 SQL 系列文章的第三篇文章中重点讲解。

4、SQL 窗口函数：前面三部分基本上已经涵盖了数据分析对于 SQL 的基础要求，但是实际工作中，我们有一些比较复杂且常见的场景，使用基础语法并不能很好的解决，比如：每月销售额的同比/环比、截至每天的累计销售额、每种商品在它所属分类下的销售额排名，窗口函数就是为这些场景而生的，如果能熟练掌握窗口函数，数据分析中就基本上不会遇到什么 SQL 的问题了。这部分内容作为 SQL 系列文章的压轴，会在第四篇文章中重点讲解。

另外，前面已经总结了一些数据分析中常用的 Excel 使用技巧，有兴趣的可以翻看之前的文章。

2、常用 SQL 函数分类

我们在使用 SQL 进行数据提取的过程中，由于分析需要，我们还需要对数据表中的原始数据进行特定的处理，如需要对字符串进行截取、对日期字段进行格式转换、对值字段进行算术运算等，处理成我们需要的格式。熟练地掌握常见的 SQL 函数可以帮助我们高效地进行数据的处理，提高分析的效率。

数据分析中常见的 SQL 函数大致可以分为以下几类：

1、日期时间函数（用来进行日期操作的函数）

- 2、字符串函数（用来进行字符串操作的函数）
- 3、算术函数（用来进行数值计算的函数）
- 4、其他重要函数（cast/coalesce/…等）

MySQL、Oracle、SQL Server 和 Hive SQL 等不同数据库的 SQL 函数略有差异，但大部分函数是通用的，为方便讲解，我们以 MySQL 语法为例进行讲解，部分函数涉及到 Hive SQL 语法，使用的时候请注意。

3、日期时间函数

日期和时间类型的字段是数据表中最常见的字段，一般情况下，我们在分析的时候需要对日期、时间转换为我们需要的时间格式，下面这些函数可以方便地完成这些操作。

日期和时间类的函数很多，我们不需要全部记住，我们只需要记住以下几种常见场景下的函数即可。

1、日期和时间戳相互转换

如果需要返回系统当前的日期和时间，可以使用以下函数：

CURRENT_DATE：返回当前日期

CURRENT_TIME：返回当前时间

Now()：返回当前日期时间

当数据表中存储的是日期，需要转化为时间戳时，可以使用 unix_timestamp 函数，函数格式：unix_timestamp(date, format)，表示将 format 格式的日期转化为时间戳。

当数据表中存储的是时间戳，需要转化为日期时，可以使用 from_unixtime 函数，函数格式：from_unixtime(unixtime, format)，表示将时间戳转化为 format 格式的日期、

– 获取当前日期、时间 –

```
select current_date();-- 结果:2021-07-17
```

```
select current_time();-- 结果:16:36:00
```

```
select now();-- 结果: '2021-07-17 16:36:00'
```

– 日期转化为时间戳 –

```
select unix_timestamp( '2020-07-17' );-- 结果:1594915200
```

```
select unix_timestamp( '2020-07-17 16:36:00' );-- 结果:1594974960
```

– 时间戳转化为日期 –

```
select from_unixtime (1594974960); -- 结果:2020-07-17 16:36:00
select from_unixtime (1594974960,' %Y-%m-%d' ); -- 结果:20200717
select from_unixtime (1594974960,' %Y%m%d' ); -- 结果:2020-07-17
-- 日期格式如要需要转换，也可以借助时间戳来进行转换 --
select from_unixtime(unix_timestamp( '20200717' ),' %Y-%m-%d' ); -- 结果:2020-07-17
select from_unixtime(unix_timestamp( '2020-07-17' ),' %Y%m%d' ); -- 结果:20200717
```

2、日期之间相互转换

如果需要把带有时间信息的日期字段转换为只有日期的格式，我们可以使用 `date_format()` 函数，函数格式 `date_format(str,format)`，通过 `format` 将 `str` 转化为需要的日期格式；

或者使用类似的 `str_to_date(str,format)`，将字符转换成日期需要的日期格式；

当然也可以使用 `cast()` 函数进行转化，函数格式 `cast(str as date)`；

如果需要把日期转换成年/月/周等，可以使用 `year()/month()/weekofyear()` 等函数；

如果需要把日期转换成时/分/秒等，可以使用 `hour()/minute()/second()` 等函数；

– 日期格式相互转换 –

```
select date_format( '2020-07-17 16:36:00' , ' %Y-%m-%d' ); -- 结果: 2020-07-17
```

```
select str_to_date( '2020-07-17 16:36:00' , ' %Y-%m-%d' ); -- 结果: 2020-07-17
```

```
select cast( '2020-07-17 16:36:00' as date); -- 结果: 2020-07-17
```

– 日期转换为年、月、周、时、分、秒 –

```
select year( '2020-07-17 16:36:00' ); -- 结果: 2020
```

```
select month( '2020-07-17 16:36:00' ); -- 结果: 7
```

```
select weekofyear( '2020-07-17 16:36:00' ); -- 结果: 17
```

```
select hour( '2020-07-17 16:36:00' ); -- 结果: 16
```

```
select minute( '2020-07-17 16:36:00' ); -- 结果: 36
```

```
select second( '2020-07-17 16:36:00' ); -- 结果: 0
```

3、日期之间的加减操作

通常我们需要选取某段时间的数据进行分析，这个时候就可以通过日期的加减操作来实现，常用的函数主要是 `datesub/date_add/datediff` 函数，主要语法及功能如下：

`date_sub (str, INTERVAL N DAY/WEEK/MONTH)` : 返回开始日期 strN 天/周/月前的日期

`date_add(str, INTERVAL N DAY/WEEK/MONTH)` : 返回开始日期 str 增加 N 天/周/月后的日期

`datediff(enddate, startdate)` : 返回结束日期减去开始日期的天数

需要注意的是：

1、这里展示的 `date_add/date_sub` 的语法是 MySQL 的函数语法，INTERVAL 不仅可以指定 N 天的时间间隔，还可以指定 N 周/月的时间间隔，如下例中所示。在其他数据库中，这 2 个函数的语法略有差异，需要留意。

2、`datediff` 函数只能处理 'yyyy-MM-dd' 这种格式的日期，如果日期形式是 'yyyyMMdd' 的话，需要用上面提到的日期和日期格式之间的转换方法来做处理~

```
select date_add('2021-07-17', INTERVAL 7 DAY);-- 7 天后的日期, 结果:2021-07-24

select date_add('2021-07-17', INTERVAL 1 WEEK);-- 一周后的日期, 结果:2021-07-24

select date_sub('2021-07-17', INTERVAL 30 DAY); - 30 天前的日期, 结果:2021-06-17

select date_sub('2021-07-17', INTERVAL 1 MONTH); - 1 月前的日期, 结果:2021-06-17

select datediff('2021-07-17', '2021-07-01'); - 结果:16
```

4、字符串函数

除日期时间函数以外, 对于数据库中的字符串字段我们也需要进行拼接、截取、替换、解析等操作, 数据分析中常用的字符串函数主要如下:

concat (str1, str2, str3): 将多个字符串首尾连接后返回

concat_ws(separator,str1,str2,...): 将多个字符串按照 separator 首尾连接

substr(str,start,length): 截取字符串 str,从 start 开始的 length 个字符

left(str,length): 截取字符串 str,从最左边开始的 length 个字符

right(str,length): 截取字符串 str,从最右边开始的 length 个字符

replace (str , str_before , str_after): 将字符串 str 中所有的 str_before 用 str_after 替换

ltrim(str)/ rtrim(str)/ trim(str) 把字符串头部/尾部/全部的空格去掉。

len(str): 返回某个字段长短

lower(str): 将字符全部小写

upper(str): 将字符全部大写

需要注意的是:

```
select split( 'This-is-MySQL' , ' -' ); – 结果 : [This,is,MySQL]
```

5、算术类函数

在数据分析中，对于数据表中的数值型字段，有时需要进行一定的算术处理，常见的算术函数有三个，它们分别是：

- ABS(num):取绝对值
- MOD (num1,num2):取余数，接受两个参数，第一个为数值，第二个为除数
- ROUND(num,N):保留小数位数，接受两个参数，第一个为数值，第二个是保留的小数位数。

一般而言，数值运算函数用的不多，但是 ROUND 在数据分析的时候还是非常实用的。

示例如下：

```
SELECT ABS(-7.21); – 取绝对值，结果：7.21;
```

```
SELECT MOD(23,12); – 23 除以 12 的余数，结果：11;
```

```
SELECT ROUND(99.865,1); – 保留一位小数 结果：99.9;
```

6、其他重要语法

在实际的数据分析中，除上述用到的函数外，还要一些使用地更为经常的函数和谓词等语法，但是有些在 Hive SQL 中可用，在 MySQL 中并不支持，主要有以下几种：

cast

cast (data as datatype) :将数据 data 转换为 datatype 类型

– 将字符串转换为数字

```
select product_id,cast(product_id as signed) as product_id_int from product;
```

结果如下：

product_id	product_id_int
0001	1
0002	2
0003	3
0004	4
0005	5
0006	6
0007	7
0008	8
0009	9
0001	1

coalesce

coalesce(data1,data2,data3.....):按照由前到后的顺序返回第一个非空值

– 取第一个非空值

select coalesce(NULL,NULL, 'first not null');

结果如下：

coalesce(NULL,NULL,'first not null')
first not null

between

between A and B:筛选位于 A 和 B 之间的数据，包含 A 和 B

– 筛选出售价在 5000-10000 的销售记录

select * from product where sale_price between 5000 and 10000 ;

结果如下：

product_id	product_name	product_category	sale_price	cost_price	sale_date
0001	iPhone	手机	8000	6500	2021-07-01
0002	MacBook Pro	电脑	9500	8000	2021-07-01
0003	HUAWEI Mate40 Pro	手机	6000	4800	2021-07-01
0004	索尼电视机	电视机	9000	6800	2021-07-01
0005	TCL电视机	电视机	6800	5000	2021-07-01
0006	创维电视机	电视机	5000	3000	2021-07-01
0009	XiaoMi 11 Pro	手机	6000	4800	2021-07-01
0001	iPhone	手机	8000	6500	2021-07-02
0002	MacBook Pro	电脑	9500	8000	2021-07-02
0003	HUAWEI Mate40 Pro	手机	6000	4800	2021-07-02

in/not in

in/not in ('cond1' , 'cond2' , 'cond3'):筛选某个字段满足候选条件中的任意一个的记录

– 筛选出手机、电脑的销售记录

select * from product where product_category in ('手机' , '电脑');

结果如下：

product_id	product_name	product_category	sale_price	cost_price	sale_date
0001	iPhone	手机	8000	6500	2021-07-01
0002	MacBook Pro	电脑	9500	8000	2021-07-01
0003	HUAWEI Mate40 Pro	手机	6000	4800	2021-07-01
0008	联想笔记本	电脑	4000	3000	2021-07-01
0009	XiaoMi 11 Pro	手机	6000	4800	2021-07-01
0001	iPhone	手机	8000	6500	2021-07-02
0002	MacBook Pro	电脑	9500	8000	2021-07-02
0003	HUAWEI Mate40 Pro	手机	6000	4800	2021-07-02
0003	HUAWEI Mate40 Pro	手机	6000	4800	2021-07-03
0002	MacBook Pro	电脑	9500	8000	2021-07-04

is null/is not null

col is null /is not null : 筛选某个字段 col 为空/不为空的记录

– 筛选出商品名称不为空的用户

select * from product where product_name is not null;

结果如下：

product_id	product_name	product_category	sale_price	cost_price	sale_date
0001	iPhone	手机	8000	6500	2021-07-01
0002	MacBook Pro	电脑	9500	8000	2021-07-01
0003	HUAWEI Mate40 Pro	手机	6000	4800	2021-07-01
0004	索尼电视机	电视机	9000	6800	2021-07-01
0005	TCL电视机	电视机	6800	5000	2021-07-01
0006	创维电视机	电视机	5000	3000	2021-07-01
0007	小米电视机	电视机	3800	2500	2021-07-01
0008	联想笔记本	电脑	4000	3000	2021-07-01
0009	XiaoMi 11 Pro	手机	6000	4800	2021-07-01
0001	iPhone	手机	8000	6500	2021-07-02

like

like 按照模糊匹配，筛选某个字段符合某种匹配规则的记录

– 筛选出包含"电视"的销售记录,%代表多个字符，更多用法参考正则表达式

select * from product where product_name like '%电视%';

结果如下：

product_id	product_name	product_category	sale_price	cost_price	sale_date
0004	索尼电视机	电视机	9000	6800	2021-07-01
0005	TCL电视机	电视机	6800	5000	2021-07-01
0006	创维电视机	电视机	5000	3000	2021-07-01
0007	小米电视机	电视机	3800	2500	2021-07-01
0004	索尼电视机	电视机	9000	6800	2021-07-02
0004	索尼电视机	电视机	9000	6800	2021-07-03
0005	TCL电视机	电视机	6800	5000	2021-07-03
0006	创维电视机	电视机	5000	3000	2021-07-03
0004	索尼电视机	电视机	9000	6800	2021-07-04
0004	索尼电视机	电视机	9000	6800	2021-07-04

case when

case data when condition1 then result1 else result2 end:对 data 进行判断，满足条件 condition1 就返回 result1，反之返回 result2

– 判断售价是否高于 5000

```
select  
sale_price,  
case  
when sale_price between 0 and 5000 then '<=5000' < span="">  
else '>5000'  
end as is_above_5000  
from product;
```

结果如下：

sale_price	is_above_5000
8000	>5000
9500	>5000
6000	>5000
9000	>5000
6800	>5000
5000	<=5000
3800	<=5000
4000	<=5000
6000	>5000
8000	>5000

一行/多行转换

多行合并为一行

对于 product 表中的销售记录，由于一个商品在每天都有可能会被销售，所以一个商品会存在着多个售价的记录，如下图所示：

product_id	product_name	sale_price
0001	iPhone	8000
0002	MacBook Pro	9500
0003	HUAWEI Mate40 Pro	6000
0004	索尼电视机	9000
0005	TCL电视机	6800
0006	创维电视机	5000
0007	小米电视机	3800
0008	联想笔记本	4000
0009	XiaoMi 11 Pro	6000
0001	iPhone	8000

如果我们想一个商品的售价汇总成一条记录，将多行转换成一行，就可以进行如下操作：

– 多行合并一行，一个商品所有的售价合并成一行，HiveSQL 语法

create table col_row_convert as – 供后面一行转多行使用

select product_id,product_name,

concat_ws(' ',collect_list(sale_price)) as sale_price_all

from product

group by product_id,product_name;

select * from col_row_convert;

结果如下：

product_id	product_name	sale_price_all
0001	iPhone	8000,8000
0002	MacBook Pro	9500,9500,9500
0003	HUAWEI Mate40 Pro	6000,6000,6000,6000
0004	索尼电视机	9000,9000,9000,9000,9000
0005	TCL电视机	6800,6800,6800
0006	创维电视机	5000,5000,5000
0007	小米电视机	3800,3800
0008	联想笔记本	4000,4000
0009	XiaoMi 11 Pro	6000

需要注意的是：代码中的 2 个重要函数：collect_list 和 concat_ws 是 Hive SQL 中的语法，

MySQL 暂不支持，前者负责将同一个 product_id,product_name 的汇总为一个 list，后者负责将 list 中的各个元素通过 “，” 进行连接，最终一个产品 product_id,product_name 形成了一条记录，与 collect_list 类似的还有 collect_set 函数，两者功能基本一致，只是后者会对汇总后的数据进行去重。

一行拆分为多行

相反地，如果我们已经有一个产品的一条汇总的售价记录，需要把一行记录拆分为多行，也就是进行上面步骤的逆操作，代码如下：

– 一行拆成多行，上面步骤的逆操作，HiveSQL 语法

```
select product_id, product_name, sale_price
from col_row_convert a
lateral view explode(split(sale_price_all,' ')) b AS sale_price;
```

结果如下：

product_id	product_name	sale_price
0001	iPhone	8000
0002	MacBook Pro	9500
0003	HUAWEI Mate40 Pro	6000
0004	索尼电视机	9000
0005	TCL电视机	6800
0006	创维电视机	5000
0007	小米电视机	3800
0008	联想笔记本	4000
0009	XiaoMi 11 Pro	6000
0001	iPhone	8000

同样需要注意的是，这里的 2 个重要函数：split 和 lateral view explode 也是 Hive SQL 中的函数，MySQL 暂不支持，前者是将上一步形成的汇总字符按照 “，” 进行分割，后者是将一行分为多行。

统计学

Q1、如何理解假设检验中的 P 值和显著性水平 α ?

讲显著性水平 α 我们就得提到前面文章中提到的第一类错误和第二类错误，可以翻看系列文章：【数分面试宝典】数据分析必备的统计学知识（一）

我们还是举之前谈恋爱的栗子，面前有一个男生，我们有 2 个假设：

H_0 : 一个真心爱你的男生

H_1 : 一个不是真心爱你的男生

如果 H_0 实际上成立，而你凭经验拒绝了 H_0 ，也就是说，你拒绝了一个你认为不爱你而实际上真心爱你的男生，那么你就犯了第一类“弃真”错误，也称为 α 风险，错杀了好人；

如果 H_0 实际上不成立，而你接受了 H_0 ，同样的道理，你接受了一个你感觉爱你而实际上并不爱你的男生，那么你就犯了第二类“纳伪”错误，也称为 β 风险，放走了坏人。

第一类错误和第二类错误这两个错误概率互相制约，你大我就会变小，你小我就会变大，基于保护零假设的原则，我们一般把一类错误概率固定住，让第一类错误概率不超过某个阈值（也就是 α 值），也就是我们常说的“显著性水平 α ”，即代表好人被冤枉的概率。通常情况下，我们不希望好人被冤枉，所以显著性水平 α 通常比较小。

显著性水平 α 是你冤枉好人的可能性，然而，每个人在这一点上是有分歧的，有的人希望 α 大一点儿，有的人希望 α 小一点儿（ α 越大，意味着检验越严格，我们冤枉好人的概率就越大）。

在这种情况下，我们就期望回答一个问题：对于面前的这个男生，我们不会冤枉他的最严格的检验水平，即最大的 α 是多少呢？得到了这个问题的答案，我们就可以轻松完成在任意严格程度上的检验了，即如果 α 大于这个值，那么我们就认为该男生不喜欢你，反之亦可。

而这个最大的 α ，就是我们的 P 值。只是这 2 个概念是有明显的区别的。显著性水平 α 是在每次统计检验之前人为规定的，通常取 $\alpha = 0.05$ 或 $\alpha = 0.01$ 。这表明，当做出拒绝原假设的决定时，其犯错误的可能性为 $\alpha = 0.05$ 或 $\alpha = 0.01$ ，而 P 值是根据实验结果计算得出的。如果计算出来的概率（P 值）小于这个标准（显著性水平 α ），就说明拒绝原假设错误的可能性很小，那就可以放心地拒绝原假设，认为这个男生确实是不爱你的；反之，大于这个标准则说明拒绝原假设错误的可能性较大，那还是接受原假设，也就是没有充足的证据，认为这个男生还是爱你的。

Q2、如何理解置信度与置信区间？

置信区间是我们所计算出的变量存在的范围，置信水平就是我们对于这个数值存在于我们计算出的这个范围的可信程度。

举例来讲，如果我们有 95% 的把握，让真正的数值在我们所计算的范围内，那么在这里，95% 是置信水平，而计算出的范围，就是置信区间。

如果置信度为 95%，则抽取 100 个样本来估计总体的均值，由 100 个样本所构造的 100 个区间中，约有 95 个区间包含总体均值。

Q3、如何理解极大似然估计？

利用已知的样本结果，反推最有可能（最大概率）导致这样结果的参数值。“似然”是“像这个样子”的意思。

极大似然估计，通俗理解来说，就是利用已知的样本结果信息，反推最具有可能（最大概率）导致这些样本结果出现的模型参数值！换句话说，极大似然估计提供了一种给定观察数据来评估模型参数的方法，即：“模型已定，参数未知”。

假如有一个黑箱子，里面有黑白两种颜色的球，数目多少不知，两种颜色的比例也不知。我们想知道箱子中白球和黑球的比例，但我们不能把箱子中的球全部拿出来数。现在我们可以每次任意从已经摇匀的箱子中拿一个球出来，记录球的颜色，然后把拿出来的球再放回箱中。这个过程可以重复，我们可以用记录的球的颜色来估计箱中黑白球的比例。假如在前面的一百次重复记录中，有 70 次是白球，请问箱子中白球所占的比例最有可能是多少？

很多人马上就有答案了：70%。而其后的理论支撑是什么呢？

我们假设罐中白球的比例是 p ，那么黑球的比例就是 $1-p$ 。因为每抽一个球出来，在记录颜色之后，我们把抽出的球放回了罐中并摇匀，所以每次抽出来的球的颜色服从同一独立分布。

这里我们把一次抽出来球的颜色称为一次抽样。题目中在一百次抽样中，70 次是白球的，30 次为黑球事件的概率是 $P(\text{样本结果}|\text{Model})$ 。

如果第一次抽象的结果记为 x_1 ，第二次抽样的结果记为 x_2 ...那么样本结果为 $(x_1, x_2, \dots, x_{100})$ 。这样，我们可以得到如下表达式：

$P(\text{样本结果}|\text{Model})$

$$\begin{aligned}
&= P(x_1, x_2, \dots, x_{100} | \text{Model}) \\
&= P(x_1 | \text{Model}) P(x_2 | \text{Model}) \dots P(x_{100} | \text{Model}) \\
&= p^{70} (1-p)^{30}.
\end{aligned}$$

好的，我们已经有了观察样本结果出现的概率表达式了。那么我们要求的模型的参数，也就是求的式中的 p 。

那么我们怎么来求这个 p 呢？

不同的 p ，直接导致 $P(\text{样本结果} | \text{Model})$ 的不同。

好的，我们的 p 实际上是有无数多种分布的。如下：

图片

那么求出 $p^{70}(1-p)^{30}$ 为 7.8×10^{-31}

p 的分布也可以是如下：

图片

那么也可以求出 $p^{70}(1-p)^{30}$ 为 2.95×10^{-27}

那么问题来了，既然有无数种分布可以选择，极大似然估计应该按照什么原则去选取这个分布呢？

答：采取的方法是让这个样本结果出现的可能性最大，也就是使得 $p^{70}(1-p)^{30}$ 值最大，那么我们就可以看成是 p 的方程，求导即可！

那么既然事情已经发生了，为什么不让这个出现的结果的可能性最大呢？这也就是最大似然估计的核心。

我们想办法让观察样本出现的概率最大，转换为数学问题就是使得：

$p^{70}(1-p)^{30}$ 最大，这太简单了，未知数只有一个 p ，我们令其导数为 0，即可求出 p 为 70%，与我们一开始认为的 70% 是一致的。其中蕴含着我们的数学思想在里面。

Q4、详细介绍一种非参数统计方法，并叙述其优缺点

非参数统计：对总体的分布不作假设或仅作非常一般性假设条件下的统计方法。

机器学习：决策树，随机森林，SVM；

假设检验：符号，符号秩，秩和检验

优点：

非参数统计方法要求的假定条件比较少，因而它的适用范围比较广泛。

多数非参数统计方法要求的思想与运算比较简单，可以迅速完成计算取得结果。

缺点：

由于方法简单，用的计量水准较低，因此，如果能与参数统计方法同时使用时，就不如参数统计方法敏感。若为追求简单而使用非参数统计方法，其检验功效就要差些。这就是说，在给定的显著性水平下进行检验时，非参数统计方法与参数统计方法相比，第Ⅱ类错误的概率 β 要大些。

对于大样本，如不采用适当的近似，计算可能变得十分复杂。

Q5、如何简单理解过拟合？如何防止过拟合？

好比你想找个女朋友。

你可能会先找你表妹问她喜欢什么，表妹说她喜欢阳光干净的男生，还说她喜欢王力宏，喜欢火锅，喜欢日料，七七八八合计一百个爱好。你规规矩矩地按照这个标准学训练自己，终于符合表妹的一切要求，完美零误差，训练完成，超级自信准备出去试试追个妹子。

可是换了个妹子，发现学到的完全没用。第二个妹子只要你阳光干净。剩下的她都不 care，她甚至讨厌王力宏，那后面的那些只会增加误差。这就事过拟合了。

怎么防止过拟合呢？应该用 cross validation，交叉比对。解释起来就是，你在你表妹那儿学到的东西，在你表姐那儿测试一下对不对。在你表姐那儿学到的，在你女同学那儿测试一下。来来回回用不同的测试对象和训练对象做交叉比对。这样学到规律就不会过拟合啦。

Q5 扑克牌 54 张，平均分成 2 份，求这 2 份都有 2 张 A 的概率。

● M 表示两个牌堆各有 2 个 A 的情况： $M=4(25!25!)$

N 表示两个牌堆完全随机的情况： $N=27!27!$

所以概率为： $M/N = 926/53 \times 172$ 。男生点击率增加，女生点击率增加，总体为何减少？

● 因为男女的点击率可能有较大差异，同时低点击率群体的占比增大。

如原来男性 20 人，点击 1 人；女性 100 人，点击 99 人，总点击率 $100/120$ 。

现在男性 100 人，点击 6 人；女性 20 人，点击 20 人，总点击率 $26/120$ 。

即那个段子“A 系中智商最低的人去读 B，同时提高了 A 系和 B 系的平均智商。”3. 参数估计用样本统计量去估计总体的参数。

可参考

<https://blog.csdn.net/liuyuemaicha/article/details/52497512>

blog.csdn.net/liuyuemaicha/article/details/52497512

Q6 假设检验

参数估计和假设检验是统计推断的两个组成部分，它们都是利用样本对总体进行某种推断，但推断的角度不同。

参数估计讨论的是用样本估计总体参数的方法，总体参数 μ 在估计前是未知的。

而在假设检验中，则是先对 μ 的值提出一个假设，然后利用样本信息去检验这个假设是否成立。

可参考

统计学假设检验中 p 值的含义具体是什么？ 3778 赞同 · 164 评论回答

Q7 置信度、置信区间

置信区间是我们所计算出的变量存在的范围，之心水平就是我们对于这个数值存在于我们计算出的这个范围的可信程度。

举例来讲，有 95% 的把握，真正的数值在我们所计算的范围内。

在这里，95% 是置信水平，而计算出的范围，就是置信区间。

如果置信度为 95%，则抽取 100 个样本来估计总体的均值，由 100 个样本所构造的 100 个区间中，约有 95 个区间包含总体均值。

可参考

如何通俗地解释「置信区间」和「置信水平」？

www.360doc.com/content/18/0317/16/15033922_737796626.shtml

Q8. 协方差与相关系数的区别和联系。

协方差：

协方差表示的是两个变量的总体的误差，这与只表示一个变量误差的方差不同。如果两个变量的变化趋势一致，也就是说如果其中一个大于自身的期望值，另外一个也大于自身的期望值，那么两个变量之间的协方差就是正值。如果两个变量的变化趋势相反，即其中一个大于自身的期望值，另外一个却小于自身的期望值，那么两个变量之间的协方差就是负值。

相关系数：

研究变量之间线性相关程度的量，取值范围是 $[-1, 1]$ 。相关系数也可以看成协方差：一种剔除了两个变量量纲影响、标准化后的特殊协方差。

可参考

如何通俗易懂地解释「协方差」与「相关系数」的概念？（转）_至臻思想_新浪博客

blog.sina.com.cn/s/blog_6aa3b1010102xkp5.html

Q9. 中心极限定理

- 中心极限定理定义：

(1) 任何一个样本的平均值将会约等于其所在总体的平均值。

- (2) 不管总体是什么分布，任意一个总体的样本平均值都会围绕在总体的平均值周围，并且呈正态分布。中心极限定理作用：

(1) 在没有办法得到总体全部数据的情况下，我们可以用样本来估计总体。

(2) 根据总体的平均值和标准差，判断某个样本是否属于总体。可参考

怎样理解和区分中心极限定理与大数定律？ 1435 赞同 · 69 评论回答

Q10. p 值的含义。

- 基本原理只有 3 个： 1、一个命题只能证伪，不能证明为真 2、在一次观测中，小概率事件不可能发生 3、在一次观测中，如果小概率事件发生了，那就是假设命题为假

- 证明逻辑就是：我要证明命题为真->证明该命题的否命题为假->在否命题的假设下，观察到小概率事件发生了->否命题被推翻->原命题为真->搞定。

结合这个例子来看：证明 A 是合格的投手-》证明“A 不是合格投手”的命题为假-》观察到一个事件（比如 A 连续 10 次投中 10 环），而这个事件在“A 不是合格投手”的假设下，概率为 p ，小于 0.05->小概率事件发生，否命题被推翻。可以看到 p 越小-》这个事件越是小概率事件-》否命题越可能被推翻-》原命题越可信

--

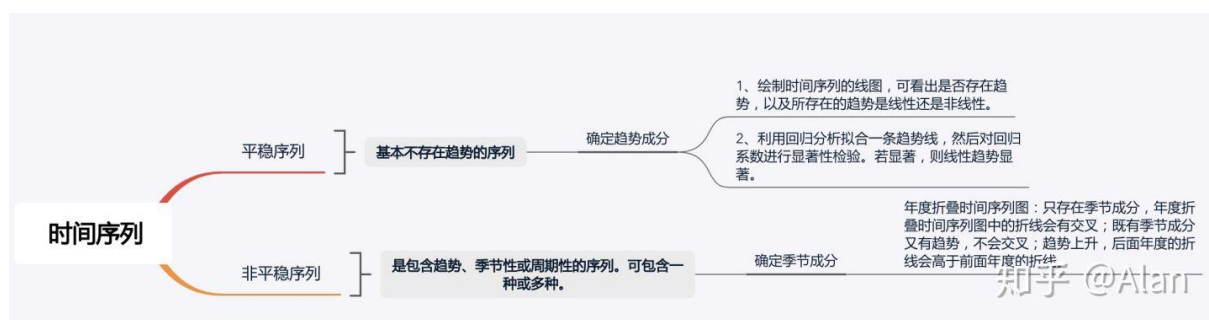
作者：吉米多维奇

链接：

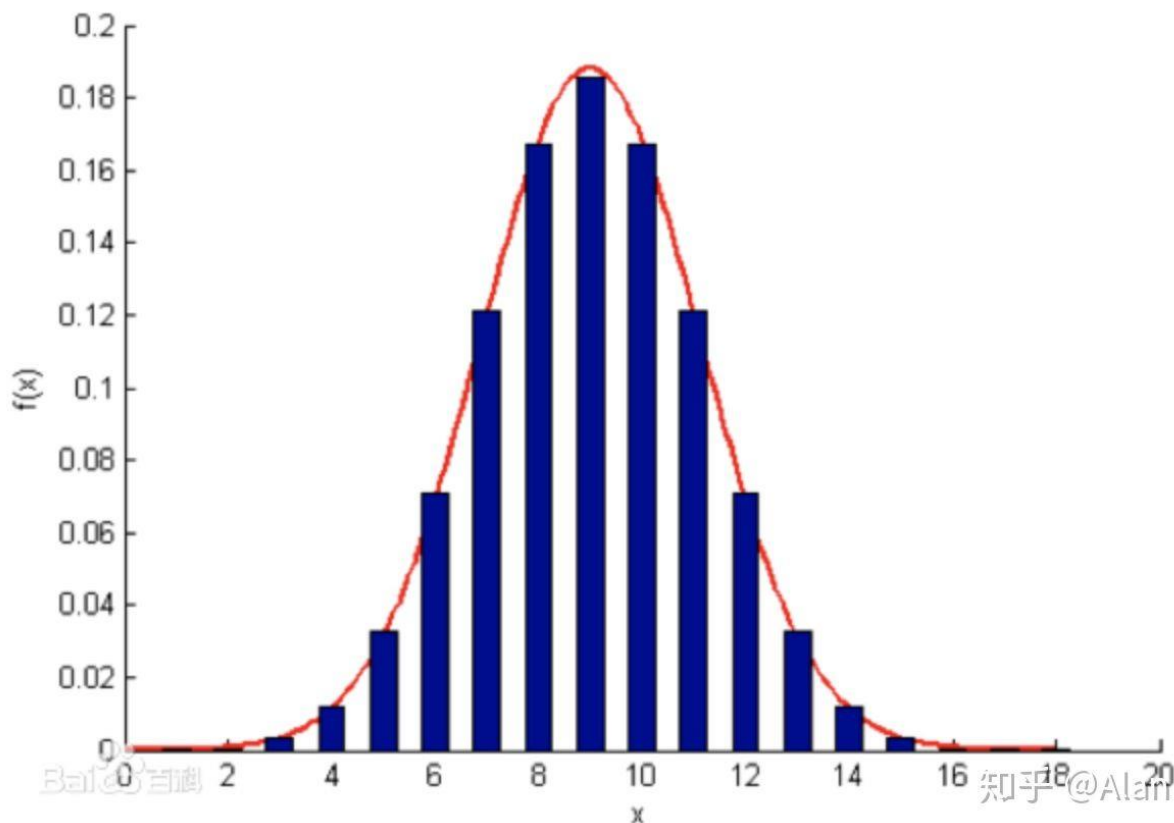
统计学假设检验中 p 值的含义具体是什么？ 267 赞同 · 22 评论回答

Q11 时间序列分析

是同一现象在不同时间上的相继观察值排列而成的序列。



Q12 怎么向小孩子解释正态分布



(随口追问了一句小孩子的智力水平，面试官说七八岁，能数数)

- 拿出小朋友班级的成绩表，每隔 2 分统计一下人数（因为小学一年级大家成绩很接近），画出钟形。然后说这就是正态分布，大多数的人都集中在中间，只有少数特别好和不够好
- 拿出隔壁班的成绩表，让小朋友自己画画看，发现也是这样的现象
- 然后拿出班级的身高表，发现也是这个样子的
- 大部分人之间是没有太大差别的，只有少数人特别好和不够好，这是生活里普遍看到的现象，这就是正态分布

Q13、下面对于“预测变量间可能存在较严重的多重共线性”的论述中错误的是？

- A. 回归系数的符号与专家经验知识不符（对）
- B. 方差膨胀因子（VIF）<5（错，大于 10 认为有严重多重共线性）

C. 其中两个预测变量的相关系数 ≥ 0.85 （对）

D. 变量重要性与专家经验严重违背（对）

Q14. PCA 为什么要中心化？PCA 的主成分是什么？

- 因为要算协方差。
- 单纯的线性变换只是产生了倍数缩放，无法消除量纲对协方差的影响，而协方差是为了让投影后方差最大。在统计学中，主成分分析（PCA）是一种简化数据集的技术。它是一个线性变换。这个变换把数据变换到一个新的坐标系中，使得任何数据投影的第一大方差在第一个坐标(称为第一主成分)上，第二大方差在第二个坐标(第二主成分)上，依次类推。主成分分析经常用减少数据集的维数，同时保持数据集的对方差贡献最大的特征。这是通过保留低阶主成分，忽略高阶主成分做到的。这样低阶成分往往能够保留住数据的最重要方面。但是，这不是一定的，要视具体应用而定。

主成分分析的原理是设法将原来变量重新组合成一组新的相互无关的几个综合变量，同时根据实际需要从中可以取出几个较少的综合变量尽可能多地反映原来变量的信息的统计方法叫做主成分分析或称主分量分析，也是数学上处理降维的一种方法。主成分分析是设法将原来众多具有一定相关性（比如 P 个指标），重新组合成一组新的互相无关的综合指标来代替原来的指标。通常数学上的处理就是将原来 P 个指标作线性组合，作为新的综合指标。最经典的做法就是用 F_1 （选取的第一个线性组合，即第一个综合指标）的方差来表达，即 $Va(rF_1)$ 越大，表示 F_1 包含的信息越多。因此在所有的线性组合中选取的 F_1 应该是方差最大的，故称 F_1 为第一主成分。如果第一主成分不足以代表原来 P 个指标的信息，再考虑选取 F_2 即选第二个线性组合，为了有效地反映原来信息， F_1 已有的信息就不需要再出现再 F_2 中，用数学语言表达就是要求 $Cov(F_1, F_2) = 0$ ，则称 F_2 为第二主成分，依此类推可以构造出第三、第四，……，第 P 个主成分。

13. 极大似然估计

利用已知的样本结果，反推最有可能（最大概率）导致这样结果的参数值。

机器学习

1. 给你一个无序数组，怎么才能合理采样？

- 无序数组是相对有序数组而言的，无序数组并不等于随机，我们要做的是将无序数组洗

牌，得到随机排列。

对于无序数组， n 个元素能产生 $n!$ 种排序。如果洗牌算法能产生 $n!$ 种不同的结果，并且这些结果产生的概率相等，那么这个洗牌算法是正确的。

方法：for i in range(len(n)): swap(arr[i], arr[random(i,n)])

这段代码是对随机确定数组第一位的值，然后递归对剩余的数组进行相同的过程，可以产生 $n!$ 中等可能的排序情况。参考资料：

<https://blog.csdn.net/rtian001/article/details/50348999>

blog.csdn.net/rtian001/article/details/50348999

https://blog.csdn.net/jiang_zzz/article/details/53786999

blog.csdn.net/jiang_zzz/article/details/53786999

2. 常用的 Python 库有哪些？

- numpy: 矩阵运算
- sklearn: 常用机器学习和数据挖掘工具库
- scipy: 基于 numpy 做高效的数学计算，如积分、线性代数、稀疏矩阵等
- pandas: 将数据用表的形式进行操作
- matplotlib: 数据可视化工具
- seaborn: 数据可视化工具
- keras/tensorflow/theano: 深度学习工具包
- NLTK: 自然语言处理工具包
- beautifulsoap: 网页文档解析工具

3. 行存储和列存储的区别。(列式数据库,更符合人类阅读习惯)

- (1) 行存储: 传统数据库的存储方式，同一张表内的数据放在一起，插入更新很快。缺点是每次查询即使只涉及几列，也要把所有数据读取。

(2) 列存储: OLAP 等情况下，将数据按照列存储会更高效，每一列都可以成为索引，投影很高效。缺点是查询是选择完成时，需要对选择的列进行重新组装。

“当你的核心业务是 OLTP 时，一个行式数据库，再加上优化操作，可能是个最好的选择。

当你的核心业务是 OLAP 时，一个列式数据库，绝对是更好的选择”参考：

https://blog.csdn.net/qq_26091271/article/details/51778675

blog.csdn.net/qq_26091271/article/details/51778675

;

为什么列存储数据库读取速度会比传统的行数据库快? 240 关注 · 13 回答问题

4.K-Means 算法原理及改进，遇到异常值怎么办？评估算法的指标有哪些？

- k-means 原理：
- 在给定 K 值和 K 个初始类簇中心点的情况下，把每个点(亦即数据记录)分到离其最近的类簇中心点所代表的类簇中，所有点分配完毕之后，根据一个类簇内的所有点重新计算该类簇的中心点(取平均值)，然后再迭代的进行分配点和更新类簇中心点的步骤，直至类簇中心点的变化很小，或者达到指定的迭代次数。改进：

a. kmeans++：初始随机点选择尽可能远，避免陷入局部解。方法是 n+1 个中心点选择时，对于离前 n 个点选择到的概率更大

b. mini batch kmeans：每次只用一个子集做重入类并找到类心（提高训练速度）

c. ISODATA：对于难以确定 k 的时候，使用该方法。思路是当类下的样本小时，剔除；类下样本数量多时，拆分

- d. kernel kmeans：kmeans 用欧氏距离计算相似度，也可以使用 kernel 映射到高维空间再聚类遇到异常值：

a. 有条件的话使用密度聚类或者一些软聚类的方式先聚类，剔除异常值。不过本来用 kmeans 就是为了快，这么做有些南辕北辙了

b. 局部异常因子 LOF：如果点 p 的密度明显小于其邻域点的密度，那么点 p 可能是异常值

参考：<https://blog.csdn.net/wangyibo0201/article/details/51705966>

blog.csdn.net/wangyibo0201/article/details/51705966

c. 多元高斯分布异常点检测

d. 使用 PCA 或自动编码器进行异常点检测：使用降维后的维度作为新的特征空间，其降维结果可以认为剔除了异常值的影响（因为过程是保留使投影后方差最大的投影方向）

e. isolation forest：基本思路是建立树模型，一个节点所在的树深度越低，说明将其从样本空间划分出去越容易，因此越可能是异常值。是一种无监督的方法，随机选择 n 个 sumsampe，随机选择一个特征一个值。

参考：

<https://blog.csdn.net/u013709270/article/details/73436588>

blog.csdn.net/u013709270/article/details/73436588

f. winsorize：对于简单的，可以对单一维度做上下截取

- 评估聚类算法的指标：

- a. 外部法（基于有标注）：Jaccard 系数、纯度

- b. 内部法（无标注）：内平方和 WSS 和外平方和 BSS

- c. 此外还要考虑到算法的时间空间复杂度、聚类稳定性等

5.数据预处理过程有哪些？

- 缺失值处理：删、插

- 异常值处理

- 特征转换：时间特征 sin 化表示

- 标准化：最大最小标准化、z 标准化等

- 归一化：对于文本或评分特征，不同样本之间可能有整体上的差异，如 a 文本共 20 个词，b 文本 30000 个词，b 文本中各个维度上的频次都很可能远远高于 a 文本

- 离散化：onehot、分箱等

6. 随机森林原理？有哪些随机方法？

- 随机森林原理：通过构造多个决策树，做 bagging 以提高泛化能力

- subsample（有放回抽样）、subfeature、低维空间投影（特征做组合，参考林轩田的《机器学习基石》）

7. PCA（主成分分析）

- 主成分分析是一种降维的方法

- 思想是将样本从原来的特征空间转化到新的特征空间，并且样本在新特征空间坐标轴上的投影方差尽可能大，这样就能涵盖样本最主要的信息

- 方法：

- a. 特征归一化

- b. 求样本特征的协方差矩阵 A

- c. 求 A 的特征值和特征向量，即 $AX=\lambda X$

- d. 将特征值从大到小排列，选择 topK，对应的特征向量就是新的坐标轴（采用最大方差理论解释，参考：<https://blog.csdn.net/huang1024rui/article/details/46662195>）PCA 也可以看成激活函数为线性函数的自动编码器（参考林轩田的《机器学习基石》第 13 课，深度学习）

8. hive? spark? sql? nlp?

- Hive 允许使用类 SQL 语句在 hadoop 集群上进行读、写、管理等操作
- Spark 是一种与 hadoop 相似的开源集群计算环境，将数据集缓存在分布式内存中的计算平台，每轮迭代不需要读取磁盘的 IO 操作，从而答复降低了单轮迭代时间

10. NVL 函数

- 是 oracle 的一个函数
- NVL(string1, replace_with), 如果 string1 为 NULL，则 NVL 函数返回 replace_with 的值，否则返回原来的值

11. LR

- 用于分类问题的线性回归
- 采用 sigmoid 对输出值进行 01 转换
- 采用似然法求解
- 手推
- 优缺点局限性
- 改进空间

12. sql 中 null 与 ' ' 的区别。

- null 表示空，用 is null 判断
- "" 表示空字符串，用=""判断

13. 数据库与数据仓库的区别。

- 简单理解下数据仓库是多个数据库以一种方式组织起来
- 数据库强调范式，尽可能减少冗余
- 数据仓库强调查询分析的速度，优化读取操作，主要目的是快速做大量数据的查询
- 数据仓库定期写入新数据，但不覆盖原有数据，而是给数据加上时间戳标签
- 数据库采用行存储，数据仓库一般采用列存储（行存储与列存储区别见题 3）
- 数据仓库的特征是面向主题、集成、相对稳定、反映历史变化，存储数历史数据；数据库

是面向事务的，存储在线交易数据

- 数据仓库的两个基本元素是维表和事实表，维是看待问题的角度，比如时间、部门等，事实表放着要查询的数据

14. SQL 的数据类型。

- 字符串：char、varchar、text
- 二进制串：binary、varbinary
- 布尔类型：boolean
- 数值类型：integer、smallint、bigint、decimal、numeric、float、real、double
- 时间类型：date、time、timestamp、interval

15. 分类算法性能的主要评价指标。

- 查准率、查全率、F1
- AUC
- LOSS
- Gain 和 Lift
- WOE 和 IV

16. 数据缺失怎么办

- 删除样本或删除字段
- 用中位数、平均值、众数等填充
- 插补：同类均值插补、多重插补、极大似然估计
- 用其它字段构建模型，预测该字段的值，从而填充缺失值（注意：如果该字段也是用于预测模型中作为特征，那么用其它字段建模填充缺失值的方式，并没有给最终的预测模型引入新信息）
- onehot，将缺失值也认为一种取值
- 压缩感知及矩阵补全

17. GBDT（梯度提升树）

- 首先介绍 Adaboost Tree，是一种 boosting 的树集成方法。基本思路是依次训练多棵树，每棵树训练时对分错的样本进行加权。树模型中对样本的加权实际是对样本采样几率的加权，在进行有放回抽样时，分错的样本更有可能被抽到
- GBDT 是 Adaboost Tree 的改进，每棵树都是 CART（分类回归树），树在叶节点输出的是

一个数值，分类误差就是真实值减去叶节点的输出值，得到残差。GBDT 要做的就是使用梯度下降的方法减少分类误差值。

- 在 GBDT 的迭代中，假设我们前一轮迭代得到的强学习器是 $f_{t-1}(x)$ ，损失函数是 $L(y, f_{t-1}(x))$ ，我们本轮迭代的目标是找到一个 CART 回归树模型的弱学习器 $h_t(x)$ ，让本轮的损失损失 $L(y, f_t(x)) = L(y, f_{t-1}(x) + h_t(x))$ 最小。也就是说，本轮迭代找到决策树，要让样本的损失尽量变得更小。GBDT 的思想可以用一个通俗的例子解释，假如有个人 30 岁，我们首先用 20 岁去拟合，发现损失有 10 岁，这时我们用 6 岁去拟合剩下的损失，发现差距还有 4 岁，第三轮我们用 3 岁拟合剩下的差距，差距就只有一岁了。如果我们的迭代轮数还没有完，可以继续迭代下面，每一轮迭代，拟合的岁数误差都会减小。

参考：梯度提升树(GBDT)原理小结 - 刘建平 Pinard - 博客园

www.cnblogs.com/pinard/p/6140514.html

- 得到多棵树后，根据每颗树的分类误差进行加权投票

18. 如何避免决策树过拟合？

- 限制树深
- 剪枝
- 限制叶节点数量
- 正则化项
- 增加数据
- bagging (subsample、subfeature、低维空间投影)
- 数据增强 (加入有杂质的数据)
- 早停

19.SVM 的优缺点

- 优点：
 - a. 能应用于非线性可分的情况

b. 最后分类时由支持向量决定，复杂度取决于支持向量的数目而不是样本空间的维度，避免了维度灾难

c. 具有鲁棒性：因为只使用少量支持向量，抓住关键样本，剔除冗余样本

● d. 高维低样本下性能好，如文本分类缺点：

a. 模型训练复杂度高

b. 难以适应多分类问题

c. 核函数选择没有较好的方法论

20. 统计教授多门课老师数量并输出每位老师教授课程数统计表。

解：设表 class 中字段为 id, teacher, course

● 统计教授多门课老师数量

```
select count( * )
```

```
from class
```

```
group by teacher
```

● having count(*) > 1 输出每位老师教授课程数统计

```
select teacher, count(course) as count_course
```

```
from class
```

group by teacher

21. 有 uid, app 名称, app 类别, 数据百亿级别, 设计算法算出每个 app 类别只安装了一个 app 的 uid 总数。

● 小数据量的话直接查询：

```
select b.apptype, count(b.uid) from (select uid, apptype, count(appname) as num from app_info  
group by uid, apptype having count(appname) = 1 ) b group by b.apptype
```

● 大数据量下（没用过 hadoop 不太清楚，望大家指正）

a. 原始文件可以拼接为 uid-app-category

b. map 阶段形成的 <k,v> 是 <uid-category, 1>

c. reduce 阶段统计 key 为“uid-category”的 count 数量

d. 只保留 count 为 1 的数据

e. 剩下的数据量直接统计 uid 出现次数 = category 类别数的数据

22. 请说明随机森林较一般决策树稳定的几点原因。

- bagging 的方法，多个树投票提高泛化能力
- bagging 中引入随机（参数、样本、特征、空间映射），避免单棵树的过拟合，提高整体泛化能力

23. 什么是聚类分析？聚类算法有哪几种？请选择一种详细描述其计算原理和步骤。

- 聚类分析是一种无监督的学习方法，根据一定条件将相对同质的样本归到一个类总。
- 聚类方法主要有：
 - a. 层次聚类
 - b. 划分聚类：kmeans
 - c. 密度聚类
 - d. 网格聚类
 - e. 模型聚类：高斯混合模型 k-means 比较好介绍，选 k 个点开始作为聚类中心，然后剩下的点根据距离划分到类中；找到新的类中心；重新分配点；迭代直到达到收敛条件或者迭代次数。优点是快；缺点是要先指定 k ，同时对异常值很敏感。

24. 余弦距离与欧式距离求相似度的差别。

1. 欧氏距离能够体现个体数值特征的绝对差异，所以更多的用于需要从维度的数值大小中体现差异的分析，如使用用户行为指标分析用户价值的相似度或差异。
2. 余弦距离更多的是从方向上区分差异，而对绝对的数值不敏感，更多的用于使用用户对内容评分来区分兴趣的相似度和差异，同时修正了用户间可能存在的度量标准不统一的问题（因为余弦距离对绝对数值不敏感）。总体来说，欧氏距离体现数值上的绝对差异，而余弦距离体现方向上的相对差异。

（1）例如，统计两部剧的用户观看行为，用户 A 的观看向量为(0,1)，用户 B 为(1,0)；此时二者的余弦距很大，而欧氏距离很小；我们分析两个用户对于不同视频的偏好，更关注相对差异，显然应当使用余弦距离。

(2) 而当我们分析用户活跃度, 以登陆次数(单位: 次)和平均观看时长(单: 分钟)作为特征时, 余弦距离会认为(1,10)、(10,100)两个用户距离很近; 但显然这两个用户活跃度是有着极大差异的, 此时我们更关注数值绝对差异, 应当使用欧氏距离。25. 数据清理中, 处理缺失值的方法是?

由于调查、编码和录入误差, 数据中可能存在一些无效值和缺失值, 需要给予适当的处理。常用的处理方法有: 估算, 整例删除, 变量删除和成对删除。

1. 估算(estimation)。最简单的办法就是用某个变量的样本均值、中位数或众数代替无效值和缺失值。这种办法简单, 但没有充分考虑数据中已有的信息, 误差可能较大。另一种办法就是根据调查对象对其他问题的答案, 通过变量之间的相关分析或逻辑推论进行估计。例如, 某一产品的拥有情况可能与家庭收入有关, 可以根据调查对象的家庭收入推算拥有这一产品的可能性。

2. 整例删除(casewise deletion)是剔除含有缺失值的样本。由于很多问卷都可能存在缺失值, 这种做法的结果可能导致有效样本量大大减少, 无法充分利用已经收集到的数据。因此, 只适合关键变量缺失, 或者含有无效值或缺失值的样本比重很小的情况。

3. 变量删除(variable deletion)。如果某一变量的无效值和缺失值很多, 而且该变量对于所研究的问题不是特别重要, 则可以考虑将该变量删除。这种做法减少了供分析用的变量数目, 但没有改变样本量。

4. 成对删除(pairwise deletion)是用一个特殊码(通常是 9、99、999 等)代表无效值和缺失值, 同时保留数据集中的全部变量和样本。但是, 在具体计算时只采用有完整答案的样本, 因而不同的分析因涉及的变量不同, 其有效样本量也会有所不同。这是一种保守的处理方法, 最大限度地保留了数据集中的可用信息。